

APRENDIZAJE POR REFUERZO OFFLINE PARA LA RESOLUCIÓN DE TAREAS ROBÓTICAS

TRABAJO FIN DE GRADO
PARA LA TITULACIÓN DEL
GRADO EN ROBÓTICA

AUTOR:
SALVADOR PÉREZ LAGO

TUTOR:
Dr. FRANCISCO JAVIER GARCÍA POLO
Dpto. de Electrónica e Computación

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

TRABAJO FIN DE GRADO

**APRENDIZAJE POR REFUERZO OFFLINE
PARA LA RESOLUCIÓN DE TAREAS
ROBÓTICAS**

AUTOR:
SALVADOR PÉREZ LAGO

TUTOR:
Dr. FRANCISCO JAVIER
GARCÍA POLO
Dpto. de Electrónica e Computación

*Este trabajo se presenta para cumplir los requerimientos normativos
exigidos para conseguir el Grado en Robótica*

en la

Escola Politécnica Superior de Enxeñaría.

Fecha de la Convocatoria: Febrero de 2024



ESCOLA POLITÉCNICA SUPERIOR
DE ENXEÑARÍA

Autorización

Yo, SALVADOR PÉREZ LAGO, alumno de la titulación del Grado en Robótica de la **Escola Politécnica Superior de Enxeñaría** de la **Universidade de Santiago de Compostela**, con DNI 53.489.295 N, autorizo la ubicación de la siguiente copia de mi **Trabajo Fin de Grado** en la biblioteca de Intercentros para que pueda ser consultada por las personas que lo deseen.

Firmado: PEREZ LAGO SALVADOR 53489295N

Firmado digitalmente
por PEREZ LAGO
SALVADOR - 53489295N
Fecha: 2024.01.21
23:15:19 +01'00'

Fecha: 21 de enero de 2024

No entiendes algo realmente si solo lo entiendes de una forma.

Marvin Minsky

Resumen

APRENDIZAJE POR REFUERZO OFFLINE PARA LA RESOLUCIÓN DE TAREAS ROBÓTICAS

por
SALVADOR PÉREZ LAGO

Palabras Clave: *Aprendizaje Automático, Agentes Inteligentes, Robótica Adaptativa, Aprendizaje por Refuerzo, RL Offline, Redes Neuronales*

El aprendizaje por refuerzo offline propone aprender políticas utilizando conjuntos de datos recopilados en una interacción (segura) con el entorno físico. Estos algoritmos han hecho posible aprender políticas útiles a partir de datos que luego pueden ser implementadas en el mundo real donde las interacciones online pueden ser costosas o peligrosas, como las que existen en robótica. Sin embargo, estos agentes offline están limitados por el *distributional shift*, un problema inherente al uso de conjuntos de datos estáticos para su entrenamiento. En este trabajo de fin de grado se presenta un método que pretende mitigar el impacto de ese problema, mediante la implementación de un modelo del entorno compuesto por un ensemble de redes neuronales. Con este modelo es posible evitar, mediante el cálculo de la incertidumbre asociado a cada estado, que los agentes se salgan de la distribución de los datos que componen el dataset de entrenamiento, evitando que se alcancen situaciones potencialmente peligrosas.

Resumo

Aprendizaxe por reforzo offline para a resolución de tarefas robóticas

por
SALVADOR PÉREZ LAGO

Palabras Chave: *Aprendizaxe Automática, Axentes Intelixentes, Robótica Adaptativa, Aprendizaxe por Reforzo, RL Offline, Redes Neurais*

A aprendizaxe por reforzo offline propón aprender políticas utilizando conxuntos de datos recollidos nunha interacción (segura) co entorno físico. Estes algoritmos fixeron posible aprender políticas útiles a partir de datos que despois poden ser implementadas no mundo real, onde as interaccións online poden ser custosas ou perigosas, como na robótica. Con todo, estes axentes offline están limitados polo *distributional shift*, un problema inherente ao emprego de conxuntos de datos estáticos para o seu adestramento. Neste traballo de fin de grado expónse un método que pretende mitigar o impacto dese problema, mediante a implementación dun modelo do entorno composto por un ensemble de redes neuronais. Con este modelo faise posible evitar, mediante o cálculo da incerteza asociada a cada estado, que os axentes saian da distribución dos datos que compoñen o dataset de adestramento, evitando que chegar a situacións potencialmente perigosas.

Abstract

Offline reinforcement learning for solving robotical tasks

by

SALVADOR PÉREZ LAGO

Keywords: *Machine Learning, Intelligent Agents, Adaptative Robotics, Reinforcement Learning, Offline RL, Neural Networks*

Offline reinforcement learning aims to learn policies using datasets collected from (safe) interactions with the physical environment. These algorithms have made it possible to learn useful policies from data that can then be implemented in the real world, where online interactions can be costly or dangerous, such as in robotics. However, these offline agents are limited by *distributional shift*, an inherent problem when using static datasets for training. This project presents a method that seeks to mitigate the impact of this problem by implementing a model of the environment by using an ensemble of neural networks. With this model, it is possible to prevent agents from deviating from the distribution of the data that present in the training dataset, by calculating the uncertainty associated with each state, thus helping avoid potentially dangerous situations.

Agradecimientos

Quiero agradecer el apoyo para realizar este trabajo a mi tutor Javier, por confiar en mí, y ayudarme tanto.

Índice General

Autorización	III
Resumen	VII
Resumo	IX
Abstract	XI
Agradecimientos	XIII
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura del trabajo	2
2. Estado del arte	5
2.1. Aprendizaje por Refuerzo	5
2.1.1. Introducción al Aprendizaje por Refuerzo	5
2.1.2. Métodos de resolución	6
Programación dinámica	7
Métodos basados en el modelo	7
Métodos libres del modelo	8
2.1.3. Generalización en Aprendizaje por Refuerzo	9
Aproximación de funciones	10
Discretización del espacio	11
2.2. Aprendizaje por refuerzo <i>offline</i>	12
2.2.1. Introducción al aprendizaje por refuerzo <i>offline</i>	12
2.2.2. Aprendizaje por refuerzo <i>offline</i> basado en el modelo	13
2.2.3. Desafíos	14
2.3. OpenAI Gym	16
3. Dominio de navegación OpenAI	19
3.1. Descripción del entorno	19
3.2. Mapeado del entorno a un MDP	20
3.3. Mapeado del MDP a un entorno OpenAI Gym	21
4. Aprendizaje por refuerzo <i>offline</i> basado en el modelo con gestión de incertidumbre	23
4.1. Generación del dataset <i>offline</i>	23
4.2. Construcción del <i>ensemble</i> de modelos	25
4.3. Cálculo de la incertidumbre	26

4.4. Introducción de la incertidumbre en la función de refuerzo	28
5. Resultados	31
5.1. Experimentos de generación del dataset	31
5.2. Creación de <i>ensembles</i> offline	33
5.2.1. Grid de 5x5: distribución 1	34
5.2.2. Grid de 5x5: distribución 2	36
5.2.3. Grid de 14x14: distribución 1	38
5.2.4. Grid de 14x14: distribución 2	40
6. Conclusiones y trabajo futuro	43
A. Descripción del código	45
B. Resultados de los agentes demostradores	51
B.1. Creación de los modelos para los demostradores	51
B.2. Trayectorias de agentes demostradores	52
Bibliografía	55

Índice de figuras

2.1. Esquema básico de funcionamiento del aprendizaje por refuerzo [11]	5
2.2. RL libre de modelo <i>vs</i> RL basado en el modelo [13]	7
2.3. Diferencias entre el RL <i>online</i> (a), el RL <i>off-policy</i> (b) y el RL <i>offline</i> (c) [4]	12
2.4. Como el <i>distributional shift</i> puede afectar a una política [23]	14
2.5. Algunos de los entornos disponibles en <i>OpenAI Gym</i> [25]	16
3.1. Entorno con rejilla de dimensiones 5x5	19
3.2. Entorno con rejilla de dimensiones 14x14	19
4.1. En naranja, casillas no demostradas	27
4.2. En rojo, casillas con una incertidumbre superior a un umbral $\rho = 0.3$	27
5.1. En (A) se puede ver la configuración de mapa elegida para este experimento con los estados no demostrados en naranja y en (B), (C) y (D) la incertidumbre para cada estado para los 3 modos a comparar	32
5.2. Comparativa del rendimiento en el cálculo de la incertidumbre con distinto número de modelos en el <i>ensemble</i> para el entorno 5x5	33
5.3. En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el <i>ensemble</i> asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este <i>ensemble</i> , que evita correctamente los estados no demostrados.	35
5.4. Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)	36
5.5. En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el <i>ensemble</i> asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este <i>ensemble</i> , que evita correctamente los estados no demostrados.	37
5.6. Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)	38
5.7. En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el <i>ensemble</i> asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este <i>ensemble</i> , que evita correctamente los estados no demostrados.	39
5.8. Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)	39

5.9. En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el <i>ensemble</i> asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este <i>ensemble</i> , que evita correctamente los estados no demostrados.	41
5.10. Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)	41
B.1. Ejemplos de demostradores en el entorno de 5x5	53
B.2. Ejemplos de demostradores en el entorno de 14x14	54

Lista de Abreviaciones

RL	R einforcement L earning (Aprendizaje por refuerzo)
MDP	M arkov D ecision P rocess (Proceso de Decisión de Markov)
ECM	E rror C uadrático M edio
DQN	D eep Q - N etwork
DDQN	D ouble D eep Q - N etwork
stdD	s tandard D eviation (Desviación Estándar)

1 Introducción

1.1. Motivación

La robótica está penetrando en todas las áreas de la sociedad. Actualmente prácticamente todas las tareas que podamos imaginar podrían ser realizadas o podrían recibir ayuda para su realización por parte de un robot. Este es el caso de robots domésticos que pueden aspirar, fregar suelos, o incluso planchar la ropa, o de robots industriales que ayudan en las cadenas de montaje de diversas industrias. Cada vez más empresas se dedican o se benefician de la robótica, encontrando aplicaciones en sectores como la manufactura, la logística y la atención al cliente, entre otros. En este contexto, el aprendizaje por refuerzo [1] ha surgido como un paradigma para dotar a los robots de un comportamiento inteligente al permitirles aprender de la interacción con su entorno y adaptarse a situaciones cambiantes de manera autónoma.

Sin embargo, surgen dos problemas a la hora de aplicar el aprendizaje por refuerzo en situaciones reales. Por un lado, es necesaria una cantidad ingente de experiencia que es inviable adquirir cuando hablamos de entornos reales. Por otro lado, los agentes aprenden a través de ensayo y error, es decir, para que el robot evite una situación catastrófica tiene que experimentarla previamente. Cuando tenemos un simulador [2] [3], estos dos problemas no son tales: el proceso de aprendizaje puede durar un tiempo indefinido porque el robot simulado no sufre ningún tipo de desgaste, y si se cae siempre podemos resetear la simulación. En el mundo real, sin embargo, este proceso de prueba y error es inviable.

A causa de estos motivos surge el aprendizaje por refuerzo *offline* [4], que permite aprender políticas de comportamiento a través de un conjunto de datos con experiencias previamente grabadas. Estas experiencias pueden provenir de demostradores humanos, con el fin de evitar situaciones de riesgo en el robot real [5] [6]. Estas experiencias, además, pueden ser utilizadas *offline* para aprender una política de comportamiento evitando el proceso de aprendizaje de prueba y error en el entorno real [7]. Sin embargo, aprender políticas de comportamiento a partir de estos conjuntos de experiencias sigue siendo una línea de investigación abierta. Una de las aproximaciones más utilizadas en este contexto es aproximar un modelo con la dinámica del entorno utilizando estas experiencias [8] [9], para que luego pueda ser utilizado a modo de simulador para aprender una política de comportamiento.

El problema más importante a tener en cuenta a la hora de aprender modelos con la dinámica del entorno, y utilizarlos después como simuladores para aprender una política de comportamiento, es el conocido como *distributional shift* [10]. Para poder evitarlo, sería deseable medir la incertidumbre de las predicciones de este modelo

y actuar en consecuencia, por ejemplo, huyendo de zonas poco o nada demostradas (aquellos estados que están fuera de la distribución de datos del dataset de entrenamiento). De esta forma, se podrían obtener políticas más robustas que serían fácilmente transferibles al mundo real.

1.2. Objetivos

Por todo ello, el objetivo general de este trabajo es estudiar mecanismos para tratar la incertidumbre de modelos que aproximan la dinámica del entorno, y utilizar esta incertidumbre para aprender políticas de comportamiento más robustas. Para demostrar estas ideas y su funcionalidad, en este trabajo nos centraremos en un dominio de navegación sencillo. Este objetivo general se puede dividir en los siguientes objetivos particulares:

1. Estudio del estado del arte sobre aprendizaje por refuerzo *offline*. Durante este estudio, se prestará especial atención a las estrategias actuales que se emplean en este contexto para medir la incertidumbre.
2. Selección del dominio de experimentación. En este caso, el dominio será un dominio de navegación sencillo, tipo *grid*, construido a propósito para este proyecto como un entorno *OpenAI Gym*. Este dominio permitirá explorar y validar las estrategias propuestas de gestión de la incertidumbre.
3. Estudio y diseño de la estrategia de generación de experiencias que serán utilizadas como demostraciones en este dominio. El punto de partida de los algoritmos de aprendizaje por refuerzo *offline* es un conjunto o conjuntos con experiencias o transiciones del tipo *estado*, *acción*, *estado siguiente*, *refuerzo* obtenidas del entorno objetivo. Este conjunto o conjuntos deben ser lo más completo posible, o por lo menos abarcar zonas del espacio que permitan después el aprendizaje de políticas de comportamientos adecuadas.
4. Estudio y diseño de la estrategia de generación de modelos a partir de las experiencias generadas que permitan predecir la dinámica del entorno, y de mecanismos para medir la incertidumbre en las predicciones.
5. Integración de esta incertidumbre en la función de refuerzo para guiar al agente hacia zonas del espacio bien demostradas. Se investigará cómo la incertidumbre puede influir en la toma de decisiones del agente y cómo puede ser utilizada para mejorar la eficacia del aprendizaje.
6. Experimentación en el dominio propuesto. La experimentación se llevará a cabo con varias configuraciones de *grid* y diferentes zonas demostradas/no demostradas.

1.3. Estructura del trabajo

El presente documento se estructura en los siguientes capítulos:

1. **Introducción:** Se corresponde con este capítulo del trabajo
2. **Estado del arte:** En este capítulo se describe el marco teórico en el que se enmarca el presente trabajo, abordando conceptos fundamentales del aprendizaje

por refuerzo (RL), con un enfoque específico en RL *offline*. Se profundiza en la teoría y práctica del RL, explorando tanto los métodos libres de modelo como los basados en el modelo. Además, se justifica la elección de la librería *Gymnasium* de *OpenAI Gym* para la implementación del entorno de simulación.

3. **Aprendizaje por refuerzo basado en el modelo con gestión de incertidumbre:** Este capítulo se centra en la propuesta central del trabajo, detallando el enfoque específico de aprendizaje por refuerzo basado en el modelo con gestión de incertidumbre. Se abordan conceptos teóricos relacionados y se presentan posibles soluciones prácticas, incluyendo la modelización del entorno, la construcción del modelo y estrategias para gestionar la incertidumbre.
4. **Dominio de navegación *OpenAI Gym*:** En este capítulo, se describe en detalle el entorno de simulación creado para llevar a cabo los experimentos del trabajo. Se explican las características del entorno, el espacio de estados y acciones, así como la implementación de redes neuronales para simular la dinámica del entorno y la función de recompensa.
5. **Resultados:** En este capítulo se presentan los resultados obtenidos durante las distintas fases del proceso, junto con los análisis correspondientes de los mismos. Para respaldar los resultados se adjuntan diversas figuras que muestran gráficamente las políticas y estadísticas obtenidas por los agentes entrenados y los valores de incertidumbre asociados a cada acción y estado.
6. **Conclusiones y trabajo futuro:** En este último capítulo, se exponen las conclusiones derivadas del trabajo realizado. Además, se plantean posibles líneas de investigación futuras y mejoras para el modelo propuesto.

Como cierre del documento se puede encontrar el apéndice **Descripción del código**, donde se pueden encontrar instrucciones para la ejecución del código así como una descripción de las carpetas y programas que componen el proyecto, y el apéndice **Resultados de los agentes demostradores**, donde se incluyen resultados obtenidos durante las primeras fases del proyecto pero que fueron descartados más tarde.

2 Estado del arte

En este capítulo, se definirán los conceptos fundamentales del aprendizaje por refuerzo (RL, por sus siglas en inglés), basándonos en las definiciones del libro de texto [1]. También se realizará un análisis en profundidad de los fundamentos y conceptos que constituyen el RL *offline*, un campo relativamente nuevo y en constante evolución, así como los principales problemas que se pueden encontrar al aplicar este tipo de metodología. Por último, se explicará la librería que se ha elegido como base para la implementación del entorno, justificando los motivos que han llevado a dicha elección.

2.1. Aprendizaje por Refuerzo

2.1.1. Introducción al Aprendizaje por Refuerzo

El RL es un área del aprendizaje automático cuyo objetivo es lograr que agentes encuentren la mejor manera de resolver una tarea mediante la optimización de las recompensas que reciben del entorno después de cada acción que realizan. En general, estas tareas están formalizadas por un Proceso de Decisión de Markov (MDP).

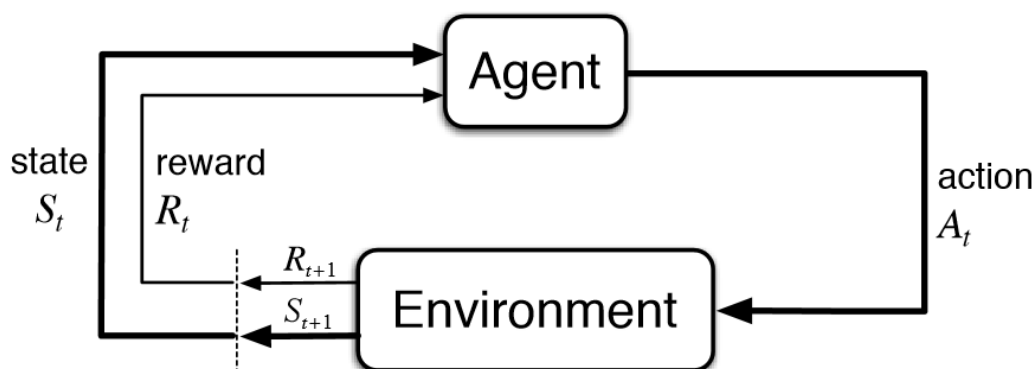


FIGURA 2.1: Esquema básico de funcionamiento del aprendizaje por refuerzo [11]

Los MDPs son representados como una tupla $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R} \rangle$ donde:

- $\mathcal{S}$ es un conjunto de estados $s \in \mathcal{S}$.
- $\mathcal{A}$ es un conjunto de acciones $a \in \mathcal{A}$.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow P(\mathcal{S})$ es una función que describe la dinámica de las transiciones del entorno. $\mathcal{T}(s_{t+1}|s_t, a_t)$ es la probabilidad de que se transite del estado s_t a s_{t+1} al ejecutar la acción a_t .

- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ es una función que define la recompensa $R(s, a)$ que se recibe después de realizar una acción a en un estado s .

El objetivo de los agentes RL es encontrar la política óptima π^* que maximice el retorno definido por la ecuación:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

donde $\gamma \in (0, 1]$ es el factor de descuento, un parámetro que representa la importancia que se da a las recompensas futuras. Cuando γ es cercano a 1, el agente prioriza recompensas a largo plazo, fomentando la toma de decisiones que benefician el futuro. Por otro lado, un γ cercano a 0 favorece la búsqueda de las recompensas más inmediatas.

La función definida por la suma de las recompensas obtenidas para cada estado $s \in \mathcal{S}$ con una política cualquiera π recibe el nombre de función de valor-estado, $V^\pi(s)$. Existe otra variante en la que se determina el valor asociado a cada una de las acciones $a \in \mathcal{A}$ que se pueden realizar en cada estado $s \in \mathcal{S}$ según una política π , la función de valor-acción $Q^\pi(s, a)$. Estas funciones son fundamentales para evaluar y comparar estados y acciones en RL.

Otro concepto importante que se debe tener en cuenta al hablar del RL, es el equilibrio entre exploración (descubrir nuevas acciones) y explotación (elegir acciones conocidas). Demasiada exploración puede llevar a pérdidas inmediatas, mientras que la explotación excesiva puede resultar en oportunidades perdidas. Estrategias como *ϵ -greedy* son comúnmente empleadas para abordar este dilema, donde el agente elige la mejor acción con probabilidad $1 - \epsilon$ y explora nuevas acciones con probabilidad ϵ .

2.1.2. Métodos de resolución

A la hora de resolver un problema de RL, el método de resolución escogido depende del conocimiento a priori que se tiene sobre el MDP. En caso de que ese conocimiento sea completo, se puede resolver usando técnicas de *programación dinámica*, que a día de hoy ya cuentan con un amplio recorrido tanto teórico como práctico [12]. En caso de que el conocimiento del MDP sea parcial o nulo, se puede escoger entre uno de estos dos métodos: *métodos basados en el modelo*, que buscan que el agente aprenda un modelo interno aproximado del MDP para después aplicar técnicas de programación dinámica, y *métodos libres de modelo*, cuyo objetivo es la resolución del problema sin ningún conocimiento del modelo.

La elección entre RL libre de modelo y basado en el modelo a menudo depende de la complejidad del entorno y los recursos computacionales disponibles. La capacidad de aprender directamente de la experiencia de los *métodos libres de modelo* puede ser ventajosa en escenarios donde la construcción de un modelo preciso es difícil. Por otro lado, los *métodos basados en el modelo* puede ser beneficioso cuando se dispone de la capacidad de construir y utilizar un modelo fiable del entorno.

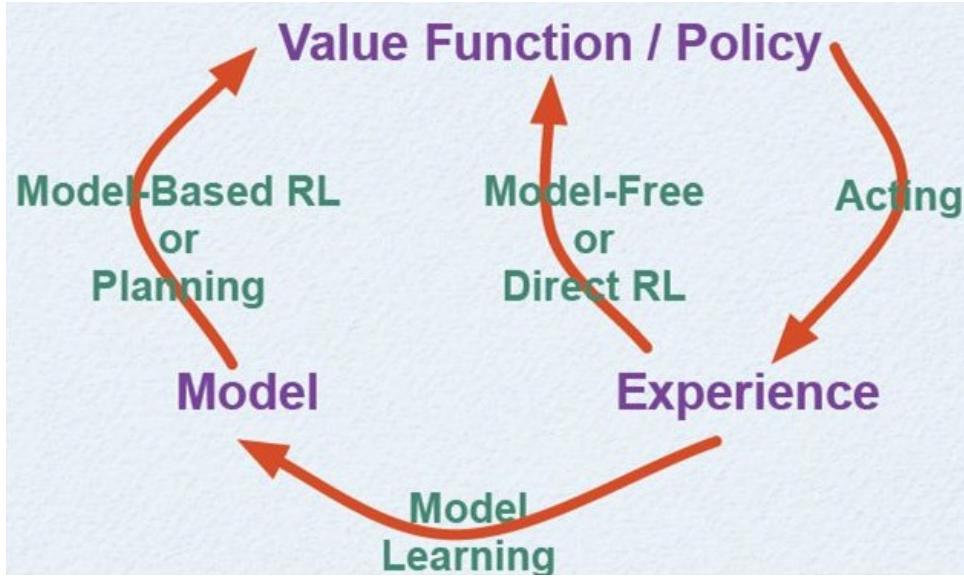


FIGURA 2.2: RL libre de modelo vs RL basado en el modelo [13]

Programación dinámica

La programación dinámica es un enfoque clave en el campo del RL, que comprende diversas técnicas y algoritmos que permiten obtener políticas de comportamiento óptimas cuando se tiene un conocimiento total del MDP. Se basa en la descomposición de un problema en subproblemas más pequeños para resolverlos de manera recursiva. De esta forma, primero se optimizan los valores de la función de valor, $V^\pi(s)$ o $Q^\pi(s, a)$, para luego poder derivar de esos resultados la política óptima π^* .

La formulación matemática de la programación dinámica se basa en las ecuaciones de Bellman, que amplían la ecuación del retorno definida en 2.1, para permitir el cálculo de la política óptima π^* a partir de un estado y una acción iniciales (s_t, a_t) cualesquiera:

$$V^*(s_t) = \max_{a_t} \sum_{s_{t+1}} \mathcal{T}(s_{t+1}|s_t, a_t) [R(s_t, a_t) + \gamma V^*(s_{t+1})] \quad (2.2)$$

$$Q^*(s_t, a_t) = \sum_{s_{t+1}} \mathcal{T}(s_{t+1}|s_t, a_t) \left[R(s_t, a_t) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1}) \right] \quad (2.3)$$

El cálculo directo de estas ecuaciones es imposible debido al carácter recursivo de las mismas, pero existen algoritmos que permiten su resolución de forma iterativa mediante la inicialización de una política aleatoria que luego va convergiendo paso a paso a políticas cada vez más próximas a π^* [14] [15]. Entre los métodos más relevantes dentro de este enfoque podemos encontrar el de *Iteración de Valor* o el de *Iteración de Política*.

Métodos basados en el modelo

Cuando no se tiene un conocimiento completo del MDP del entorno, una de las soluciones posibles consiste construir un modelo aproximado del entorno mediante

exploración. Este modelo se utiliza para simular la experiencia del agente, con el objetivo de permitirle simular la dinámica del entorno, *i.e.* hacer una predicción de las transiciones y de las recompensas que recibirá de forma anticipada, y así poder aplicar métodos de programación dinámica sobre esos conocimientos para la optimización de la política.

Para lograr esto, se busca construir un modelo $S \times A \rightarrow S$ que, dado un estado $s \in S$ y una acción $a \in A$, devuelvan el siguiente estado $s' \in S$ del sistema. Matemáticamente, podría expresarse como:

$$s' = M(s, a) \quad (2.4)$$

donde M representa la función del modelo, y su propósito es describir lo más precisamente posible las dinámicas del entorno. Este modelo M puede adoptar diversas formas, desde modelos lineales hasta complejas arquitecturas de redes neuronales. La elección del tipo de modelo depende de la complejidad de las interacciones entre el agente y el entorno.

Algunos enfoques de este tipo de RL se centran en buscar la mejora de la precisión del modelo, mientras que otros buscan estrategias eficientes de exploración para adquirir información útil. *DynaQ* [16], por ejemplo, utiliza un modelo del entorno para simular experiencias y actualizar las estimaciones de valor.

También es relevante mencionar el uso de modelos probabilísticos. Estos modelos capturan la incertidumbre inherente en el entorno y pueden ser fundamentales para la toma de decisiones seguras y robustas. Métodos como *Probabilistic Inference for Learning Control* (PILCO) [17] incorporan esta idea, modelando la transición del estado de manera probabilística para una planificación más precisa y adaptativa.

Métodos libres del modelo

Este tipo de métodos, al igual que en los basados en el modelo, asumen el desconocimiento de la dinámica del entorno, a excepción de las acciones que puede ejecutar el agente y de los estados en los que se puede encontrar. A grandes rasgos, los métodos libres de modelo aprenden únicamente a través de su propia experiencia mediante un proceso de exploración y utilizan el *feedback* de recompensa y estado (2.1) que obtienen del entorno para actualizar sus políticas de comportamiento o sus funciones de valor.

Algunas técnicas populares son los *Métodos de Monte Carlo*, que estiman la función de valor mediante el muestreo de secuencias de estados y el retorno de recompensas de las mismas, y los *Métodos de diferencia temporal* (*Temporal Difference* - TD), como SARSA [18] o *Q-Learning* [19].

Q-Learning es un algoritmo que hace uso de una tabla Q , donde cada entrada $Q(s, a)$ cuantifica la utilidad de realizar una acción de tomar cada acción a en un estado dado s . Inicialmente, esta tabla se llena con valores aleatorios, que se van actualizando mientras el agente explora el entorno mediante una ponderación de las recompensas inmediatas obtenidas y la estimación de la utilidad futura, como se puede observar en el Algoritmo 1.

Durante este proceso, es fundamental un ajuste correcto de los parámetros de tasa de aprendizaje α , que controla la influencia de la nueva información en la actualización, y del factor de descuento γ , que determina la importancia de las recompensas futuras.

A medida que el agente explora y acumula más experiencias, la tabla Q converge hacia valores que reflejan la mejor estrategia para cada estado, *i.e.* la política óptima π^* . Esta política óptima se obtiene seleccionando la acción con el valor máximo para cada estado en la tabla Q .

Algoritmo 1: Q-Learning

Input: Espacio de estados S

Espacio de acciones A

Un número máximo de episodios a ejecutar K

Un número máximo de pasos por episodio H

Los parámetros de tasa de aprendizaje α y factor de descuento γ

Output: $Q(s, a)$

Se inicializan la tabla Q con valores aleatorios, $\forall s \in S$ y $\forall a \in A : Q(s, a) \in \mathbb{R}$;

// Para cada episodio:

for $k = 1$ **to** K **do**

 Seleccionar s_1 ;

 // Para cada paso:

for $h = 1$ **to** H **do**

 Seleccionar acción a_h usando la política derivada de Q con algún tipo de exploración;

 Ejecutar la acción a_h u obtener el refuerzo r_{h+1} y el estado siguiente s_{h+1} ;

 Actualizar la entrada de la tabla $Q(s_h, a_h)$:

$$Q(s_h, a_h) \leftarrow Q(s_h, a_h) + \alpha \left[r_{h+1} + \gamma \max_{a'} Q(s_{h+1}, a') - Q(s_h, a_h) \right]$$

return $Q(s, a)$;

Además, según el uso que se haga de la política que se está aprendiendo, los métodos libres de modelo se pueden clasificar estos métodos en dos grupos:

- *Métodos on-policy*: Usan para su interacción con el entorno la misma política que se está aprendiendo. Es decir, se evalúa y mejora la misma política con la que se deciden las acciones del agente.
- *Métodos off-policy*: La política de interacción con el entorno es independiente de la que se está aprendiendo. Es decir, se evalúa y mejora una política distinta a la que se usa para decidir las acciones del agente durante la exploración.

2.1.3. Generalización en Aprendizaje por Refuerzo

En las secciones previas donde se han explicado los distintos métodos existentes de aprendizaje por refuerzo, se ha supuesto la existencia de un conjunto finito de estados y acciones. Sin embargo, esto último representa una situación ideal que, en la

mayoría de los casos, no es verdadera. En muchos dominios, los estados y las acciones se representan como conjuntos de atributos con valores continuos, lo que los hace intratables a nivel de cómputo. Ante esta circunstancia, es necesario buscar una representación alternativa tanto para el espacio de estados como para el de acciones. En este contexto, existen dos enfoques principales: la *Aproximación de funciones* y la *Discretización del espacio*.

Aproximación de funciones

La aproximación de funciones es fundamental para extender el aprendizaje por refuerzo a problemas de gran escala. Este enfoque recibe su nombre porque, dado un conjunto de ejemplos de la función objetivo que se quiere aprender (e.g. la función de valor-estado $V(s)$), intenta generalizar esos ejemplos para construir una aproximación completa de dicha función objetivo.

En el caso de métodos de diferencia temporal, como se describe en la sección 2.1.2, se busca estimar la función de valor-estado $V(s)$ o la función de valor-acción $Q(s, a)$. La aproximación en un instante de tiempo t de la función valor-estado, $\hat{V}_t$, o de la función valor-acción, $\hat{Q}_{a_t}$, se puede representar como una función de un conjunto de parámetros representados por el vector θ_t , en lugar de representarla de manera directa. Esto implica que el valor de la función valor-estado $\hat{V}_t$ o de la función valor-acción $\hat{Q}_{a_t}$ depende únicamente del vector de parámetros θ_t , que varía en cada instante de tiempo según cómo cambian los valores contenidos en θ_t .

La idea subyacente es que la dimensión del vector θ_t es mucho menor que el número de estados, de manera que variar unos pocos parámetros del vector modificará el valor estimado de la función de valor para muchos estados. En este contexto, el problema se puede entender como un problema de aproximación de funciones en aprendizaje supervisado, y por lo tanto, se pueden emplear diversas técnicas como redes de neuronas o árboles de decisión entre otras. Cuando se busca aproximar la función $Q(s, a)$ en lugar de la función $V(s)$, basta con introducir un parámetro de entrada adicional.

Durante el proceso de estimación de funciones, es esencial decidir la medida que permita determinar si la aproximación obtenida es buena o mala. En la mayoría de los métodos supervisados, se utiliza el *error cuadrático medio* (ECM) para este propósito. En aprendizaje por refuerzo, en caso de que la función que se pretende aproximar sea $V^\pi(s)$, el error cuadrático medio se define como la diferencia entre el valor real $V^\pi(s)$ y el valor estimado $\hat{V}_t(s)$ en el instante de tiempo t , tal como se muestra en la siguiente ecuación:

$$ECM(\theta_t) = \sum_{s \in S} P(s) [V^\pi(s) - \hat{V}_t(s)]^2 \quad (2.5)$$

donde $P(s)$ es la probabilidad de que se dé el estado s , dada por la función de transiciones del entorno, y el objetivo es, por tanto, buscar el conjunto de parámetros θ_t que minimice el error cuadrático.

De forma análoga, para la función valor-acción $Q^\pi(s, a)$ se tiene la ecuación:

$$ECM(\theta_t) = \sum_{s \in S, a \in A} P(s) [Q_a^\pi(s) - \hat{Q}_a(s)]^2 \quad (2.6)$$

La aproximación de funciones puede ser lineal o no lineal. El principal inconveniente de las funciones de aproximación lineal, en comparación con las no lineales, es la selección correcta de las características, que a menudo se modela de manera *ad-hoc*, requiriendo un conocimiento sólido del dominio. Esta es una de las razones por las que los métodos cuyo uso está más extendido son los no lineales de descenso de gradiente, donde se suelen utilizar redes neuronales como aproximadores y, en los últimos años, máquinas de vectores de soporte (*support vector machines*).

Ejemplos notables de estos métodos incluyen el algoritmo *Deep Q-Network (DQN)* [20], que emplea redes neuronales profundas para aproximar la función de valor $Q^\pi(s, a)$, y el *Actor-Critic basado en el Gradiente de Política Determinista (DDPG)* [21], que combina una red para la aproximación de la política (*Actor*) y otra para la evaluación de la función de valor (*Critic*).

El algoritmo *DQN* es una técnica muy similar al *Q-Learning*, ya que también trata de optimizar la función de valor-acción $Q(s, a)$, pero en lugar de emplear una tabla Q para aproximar esta función hace uso de redes neuronales profundas. De esta manera, se consigue trabajar con entornos con espacios de estados y acciones más complejos.

Los pesos de la red neuronal también son inicializados con valores aleatorios y se van actualizando conforme el agente va interactuando y recibiendo recompensas del entorno. Al terminar, la red neuronal final es un modelo de la función Q , que tomando como entradas un estado $s \in S$ cualquiera y una de las acciones $a \in A$ posibles, permite al agente hacer uso de la política óptima π^* en cada estado simplemente comparando la cuantía de la valoración con cada una de las acciones y eligiendo aquella que sea mayor.

Además, *DQN* hace uso del llamado *replay buffer*, una memoria donde se guardan todas las experiencias pasadas del agente para que, en lugar de actualizar la red neuronal después de cada interacción con el entorno, hacerlo muestreando aleatoriamente lotes de experiencias de la memoria. Con esto se logra estabilizar el proceso de convergencia durante el aprendizaje, ya que podría verse afectado debido a la dependencia entre muestras consecutivas.

Otro algoritmo muy similar es el *Double DQN* o *DDQN* que, como se podría intuir por su nombre, se trata de una variación del *DQN* clásico pero empleando dos redes neuronales en lugar de una. Una de ellas se usa para estimar los valores estado y la otra para los valores acción, que al combinarlos dan como resultado la estimación de la función Q . La utilización de dos redes ayuda a reducir la sobreestimación de los valores Q que puede ocurrir en *DQN*. Además, *DDQN* introduce una estrategia de doble toma de decisiones, donde una red se utiliza para seleccionar la mejor acción y la otra para evaluar el valor del próximo estado, mejorando la estabilidad del aprendizaje.

Discretización del espacio

Cuando se lidia con entornos con espacios de estados y acciones continuos, la aplicación directa de métodos tradicionales puede resultar computacionalmente imposible. En esos casos, lo recomendable es aplicar técnicas de *discretización del espacio*, que consisten en dividir estos espacios continuos en regiones discretas. Esto simplifica el problema, facilitando su resolubilidad mediante métodos de aprendizaje

por refuerzo convencionales. Sin embargo, la discretización debe realizarse cautelosamente para evitar la pérdida de información crucial, lo que podría suponer una deriva importante en la solución obtenida.

La discretización implica dividir el rango posible de cada atributo en intervalos discretos de rangos más pequeños. Por ejemplo, si se tiene un atributo de velocidad continua, se podría dividir en diferentes rangos discretos, como *baja velocidad*, *velocidad media* y *alta velocidad*. Para implementar la discretización del espacio, es necesario definir la cantidad de divisiones o *bins* en las que se dividirá cada dimensión del espacio continuo. La elección de esta cantidad es un aspecto crítico para evitar la pérdida de información ya mencionada. Idealmente, se busca lograr una representación discreta que capture de manera efectiva las características relevantes del problema.

La discretización del espacio se puede realizar de muchas maneras, y la elección del método depende del problema específico y de la naturaleza de los datos. Algunas de las técnicas comunes incluyen la partición uniforme, donde se divide cada dimensión de manera uniforme, y la partición no uniforme, que adapta la discretización según la distribución de los datos.

Una vez que se ha discretizado el espacio, se pueden aplicar métodos de aprendizaje por refuerzo diseñados para trabajar con espacios de estados y acciones discretos, como *Q-Learning* y *SARSA*, ya mencionados en secciones anteriores. Estos algoritmos son capaces de manejar de manera eficiente la información discretizada y aprender políticas óptimas en entornos donde la discretización es una necesidad.

2.2. Aprendizaje por refuerzo *offline*

2.2.1. Introducción al aprendizaje por refuerzo *offline*

El RL *offline* [4], también conocido como RL a partir de datos estáticos, presenta una perspectiva única en el campo del RL. A diferencia del RL *online*, donde el agente interactúa continuamente con el entorno, el RL *offline* implica entrenar al agente utilizando datos recopilados previamente.

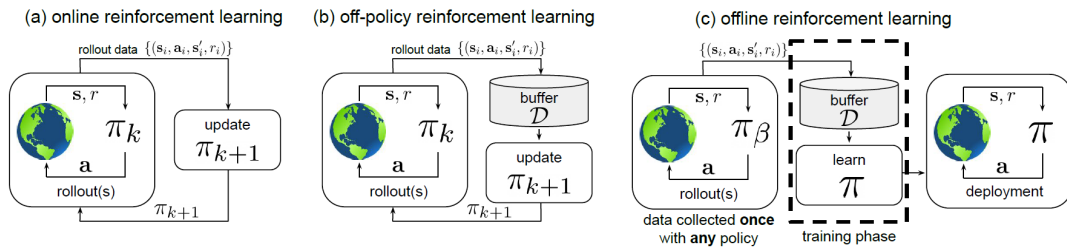


FIGURA 2.3: Diferencias entre el RL *online* (a), el RL *off-policy* (b) y el RL *offline* (c) [4]

En el RL *online*, la política π_k se va actualizando solo con los datos obtenidos por la interacción de esa misma política con el entorno, intentando converger a la política óptima π^* . En el RL *off-policy*, que es una variación del RL *online*, las experiencias obtenidas por el agente se guardan en un *buffer* de datos $\mathcal{D}$, que contendrá experiencias obtenidas por todas las políticas históricas $\pi_0, \pi_1, \dots, \pi_k$, y todos esos datos se utilizan para entrenar a la siguiente política π_{k+1} .

En cambio, en el RL *offline* se utiliza un *buffer* de datos $\mathcal{D}$ estático, *i.e.* que no pueden ser modificados durante el entrenamiento, que son recopilados de antemano con una política de comportamiento π_β (que puede ser no conocida). Durante el proceso de entrenamiento el agente no interactúa con el MDP y la política no se implementará hasta haber terminado este proceso por completo.

La estática de los datos, es decir, la falta de adquisición continua de información, introduce desafíos cuando trabajamos con entornos no estacionarios. Dado que el agente basa su entrenamiento tan solo en los datos históricos previamente recolectados, deben buscarse estrategias para conseguir algún grado de generalización frente a situaciones potencialmente cambiantes. Para intentar mitigarlo en la mayor medida posible, es recomendable procurar que $\mathcal{D}$ sea lo más grande y que contenga la mayor diversidad de datos dentro de las posibilidades.

Existen distintas estrategias a la hora de obtener los datos, pero entre las más comunes se encuentran el uso de demostradores, donde el *buffer* se rellena con datos recogidos durante episodios en los que el sistema es controlado por un humano u otro agente que ya tienen algún grado de experiencia previa en el entorno, o bien, cuando se trata de entornos la recolección de datos es demasiado costosa o complicada (*e.g.* en robótica industrial, sanitaria...) o peligrosa (*e.g.* conducción autónoma), podemos hacer uso de los miles de datasets ya existentes, bien sean de carácter público o mediante su adquisición a especialistas.

Los datos históricos pueden contener sesgos inherentes debido a que existe la posibilidad de que las elecciones de los expertos en las demostraciones no siempre sean las más adecuadas o a condiciones específicas del entorno que se den durante su recopilación. Este sesgo puede influir en la efectividad del aprendizaje de forma que imposibilite que la mejor política que puedan aprender los agentes no sea la política óptima, pero de ser este el caso, el objetivo del aprendizaje seguirá siendo el mismo, buscar la maximización del retorno obtenido por la ecuación 2.1.

2.2.2. Aprendizaje por refuerzo offline basado en el modelo

El RL *offline* basado en el modelo [22] representa una rama intrigante y prometedora dentro del amplio espectro de RL. Como se explica en 2.1.2, el objetivo principal de este tipo de técnicas es el de construir un modelo M del entorno que sea capaz de simular sus dinámicas, pero en este caso la construcción del mismo debe hacerse de manera *offline*, *i.e.* empleando un conjunto de experiencias recuperadas previamente del entorno. La elección del tipo de modelo depende de la complejidad de las interacciones entre el agente y el entorno, aunque en este caso particular las soluciones más abundantes se basan en el uso de redes neuronales, definidas por su vector paramétrico de pesos θ .

En el caso de optar por modelos lineales simples, la función M (ver Ecuación 2.4) podría representarse como una combinación lineal de las variables de entrada:

$$s' = \theta_0 + \theta_1 \cdot s + \theta_2 \cdot a \quad (2.7)$$

donde θ_0 , θ_1 , y θ_2 son parámetros del modelo que se ajustan durante el entrenamiento para minimizar la discrepancia entre los valores predichos y los reales.

En el caso de modelos no lineales, como las redes neuronales, la función M podría estar compuesta por múltiples capas, cada una con sus propios pesos y funciones de activación. La representación general para una red neuronal podría ser:

$$s' = M_{\theta}(s, a) \quad (2.8)$$

donde θ representa el vector parámetros del modelo, *i.e.* los pesos de la red neuronal. Durante el entrenamiento, estos pesos se ajustan para optimizar la capacidad del modelo para predecir el siguiente estado.

Este modelo se convierte en una herramienta esencial para realizar simulaciones *offline* e incluso para generar muestras sintéticas de experiencias del agente, ya que al predecir el siguiente estado s' para un estado s y una acción a dados, el agente es capaz de explorar posibles trayectorias y evaluar diferentes acciones sin interactuar directamente con el entorno. El emplear estas muestras como dataset para el entrenamiento permite al agente aprender de manera eficiente y segura, minimizando la necesidad de interacciones en tiempo real.

2.2.3. Desafíos

El principal reto que nos podemos encontrar al intentar resolver un problema de RL *offline* empleando modelos recibe el nombre de *distributional shift*. Este problema surge cuando la distribución de datos durante el entrenamiento difiere significativamente de la distribución real que se puede encontrar el agente al enfrentarse al entorno, lo que supondría que el agente se enfrentase a situaciones para las cuales no fue específicamente entrenado. Esto podría resultar en un rendimiento subóptimo o incluso en comportamientos inesperados, comprometiendo la aplicabilidad de la política aprendida en situaciones del mundo real. Por lo tanto, es fundamental abordar el *distributional shift* para garantizar que el modelo sea robusto y generalice de manera efectiva a nuevas situaciones.

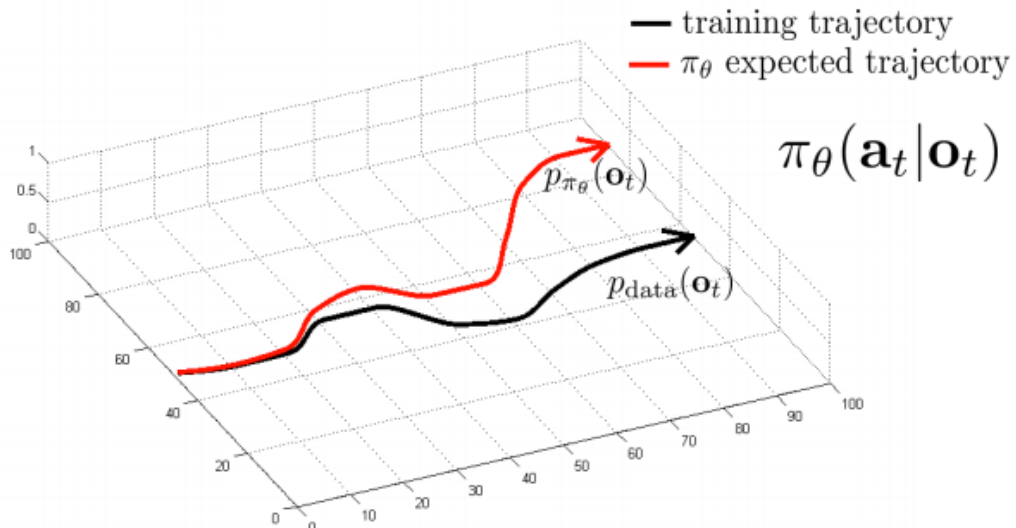


FIGURA 2.4: Como el *distributional shift* puede afectar a una política [23]

En la figura anterior 2.4 podemos ver ilustrado gráficamente lo que podría suponer el problema del *distributional shift* para un agente que se encuentre ejecutando una política de comportamiento π_θ . Si en algún momento durante su trayectoria, por el motivo que sea, llega a un estado fuera de la distribución de datos de su entrenamiento, a partir de ese momento, ya no se puede garantizar la eficacia de la política en esa nueva distribución de datos. Además, a partir del momento en que se sale de la distribución se genera un efecto de “bola de nieve”, *i.e.* que con cada acción que vaya tomando será más improbable que pueda regresar a la distribución correcta y la política será más propensa a cometer errores.

Por todo ello, la mitigación del *distributional shift* es esencial para garantizar la robustez de la política aprendida. Técnicas como la regularización, que penaliza desviaciones extremas en la distribución de datos, y métodos de ajuste de dominio, que adaptan la política a la nueva distribución, son estrategias clave para abordar este desafío. Otro enfoque posible podría ser añadir ruido a las trayectorias de entrenamiento, haciendo que las políticas que se entrenen a partir de ellas sean más robustas ante posibles desviaciones.

El segundo gran desafío del RL *offline* basado en el modelo radica en la calidad del modelo construido. Este debe describir lo más fielmente posible las complejidades del entorno para garantizar que las muestras generadas sean representativas y útiles para el aprendizaje del agente. Para lograrlo, se deben buscar estrategias que permitan validar el modelo y buscar su optimización continuada, de forma que se pueda garantizar su robustez.

En este proyecto se propone una solución que explora en profundidad esta área del RL *offline* basado en modelos. Por un lado se busca la modelización, lo más exacta posible, de la dinámica del entorno y de la función de recompensa, lo que permitirá la obtención de experiencias haciendo uso únicamente de estos modelos, sin necesidad de una interacción directa con el entorno una vez se hayan creado y validado. Por otra parte, haciendo uso de las experiencias obtenidas, se entrena un *ensemble*¹ de modelos de redes neuronales, una solución muy similar a la que podemos encontrar en MOPO [8] y, sobre todo, MOREL [9].

Con ese *ensemble* de modelos, como se explica más adelante en el Capítulo 4, es viable diferenciar entre los estados presentes en el dataset de entrenamiento y los que no, midiendo su incertidumbre. De esta forma, puede lograrse entrenar agentes que, haciendo uso de las predicciones del *ensemble*, sean capaces de encontrar soluciones en las que eviten por completo visitar aquellos estados no demostrados, o lo que es lo mismo, que están fuera de la distribución de los datos de entrenamiento, evitando efectivamente el problema del *distributional shift*.

Sin embargo, al usar redes neuronales como modelo para la representación del entorno, la selección de la arquitectura del modelo y su capacidad para generalizar se convierten en factores críticos que determinan su éxito, por lo que su diseño y *tuning* conlleva una gran dificultad. Es de especial importancia también tener en cuenta que la elección de distintos valores iniciales para el vector de parámetros del modelo θ (los pesos de la red), puede hacer que exista una cierta variación en las soluciones obtenidas, por lo que debe ser un factor a tener en cuenta.

¹Un *ensemble* es un conjunto de cosas.

2.3. OpenAI Gym

OpenAI Gym [24] se presenta como una herramienta indispensable en el ámbito del RL, proporcionando una amplia variedad de entornos estandarizados para el desarrollo, la evaluación y la comparación de algoritmos. *OpenAI Gym* ofrece una amplia variedad de entornos estandarizados que representan problemas típicos de RL, desde juegos simples hasta desafíos más complejos. Esta diversidad de entornos permite a investigadores y desarrolladores probar algoritmos en una amplia gama de escenarios, desde juegos simples hasta otros entornos más complejos.

La interfaz unificada de *OpenAI Gym* simplifica la interacción entre los agentes y los entornos. Define métodos comunes, como *reset* para reiniciar el entorno y *step* para realizar una acción, facilitando la implementación de algoritmos y la evaluación de su rendimiento.

La estructura de *OpenAI Gym* permite comparar de manera justa los algoritmos de RL. Los resultados son reproducibles, lo que facilita la replicación de experimentos y la comparación de diferentes enfoques en una variedad de entornos.

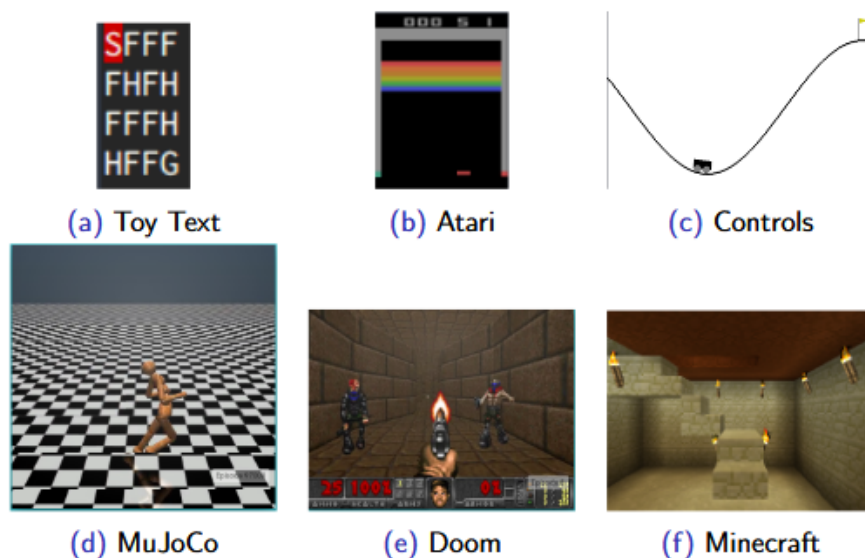


FIGURA 2.5: Algunos de los entornos disponibles en *OpenAI Gym* [25]

OpenAI Gym es compatible con el aprendizaje por refuerzo *offline*, lo que permite la utilización de datos históricos para entrenar a los agentes. Esto resulta crucial en la evaluación de políticas en situaciones del mundo real, donde la interacción continua puede ser costosa o impracticable.

Aunque *OpenAI Gym* proporciona un marco robusto para el desarrollo y la evaluación, la generalización de las políticas aprendidas en entornos simulados a situaciones del mundo real casi siempre suele ser un desafío. A la hora de migrar a entornos reales se deben aplicar estrategias cuidadosas para abordar las discrepancias entre las simulaciones y la realidad.

A pesar de haber sido desarrollada en su totalidad por la empresa *OpenAI*, esta librería ha dejado de recibir mantenimiento desde el año 2021, por lo que en este trabajo he decidido optar por emplear *Gymnasium* [26], que no es más que un fork de la librería original creado y mantenido por *The Farama Foundation*, por lo que su integración en entornos con otras actualizadas resulta mucho más sencilla.

Gymnasium mantiene la estructura general de *OpenAI Gym* y proporciona una variedad de entornos listos para usar. Su apuesta por la continuidad de actualizaciones y su soporte activo lo convierte en una elección más adecuada para proyectos que busquen ahondar en temas tan actuales como puede ser el RL *offline*.

3 Dominio de navegación OpenAI

En este capítulo, se describe el entorno creado para llevar a cabo la experimentación del trabajo. Como ya se había explicado con anterioridad en el apartado 2.3, se ha elegido la librería *Gymnasium* para la implementación del entorno de simulación.

3.1. Descripción del entorno

En este caso concreto, se ha optado por un entorno “simple”, en el sentido de que tanto el espacio de acciones como el de estados son discretos, pero no por ello la dificultad de resolver los problemas propuestos es menor ya que, como se explica más adelante en la sección 4.2 referente a la creación del modelo del entorno, se han encontrado dificultades propias de este tipo de entornos que probablemente no estarían presentes si se trabajase con espacios de estados o acciones continuos.

La propuesta presentada parte de la implementación del entorno *SimpleGrid* [27], un entorno sencillo y fácil de personalizar, consistente en un *grid* o rejilla en 2D donde un agente, cuyo estado actual se representa coloreando esa celda de azul, debe partir desde una casilla de *Inicio* (en verde) hasta la casilla de *Meta* (en rojo), navegando a través de las casillas libres de la rejilla (en blanco) y evitando colisionar contra aquellas que contienen muros (en negro) o salirse por alguno de los extremos del entorno.

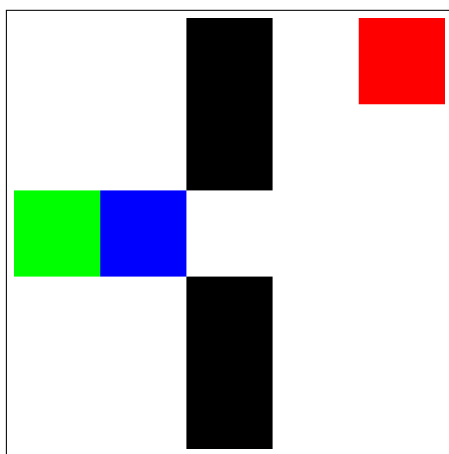


FIGURA 3.1: Entorno con rejilla de dimensiones 5x5

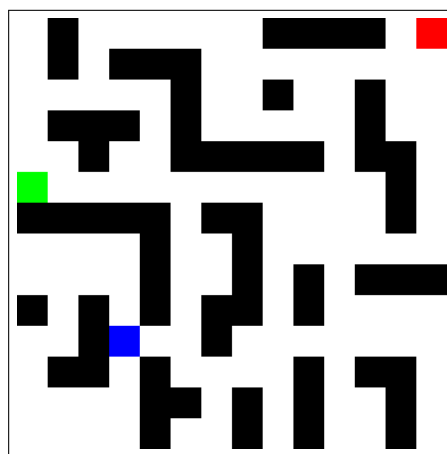


FIGURA 3.2: Entorno con rejilla de dimensiones 14x14

En bajo nivel, la rejilla está implementada como un array de arrays que contienen caracteres. El número de arrays que hay se corresponde con el número de filas (N_f) de la rejilla y el número de caracteres en cada array sería el número de columnas (N_c). No es obligatorio que la rejilla sea cuadrada, pero si que lo es que existan el mismo

número de columnas en cada fila. La distribución del mapa también es personalizable, aunque se aconseja intentar que exista al menos un camino entre el *Inicio* y la *Meta*, y que estos últimos sean únicos. Para ello, basta con cambiar el caracter de cada posición, escogiendo entre las siguientes opciones:

- “.”: casilla libre.
- “#”: casilla ocupada por un obstáculo.
- “S”: casilla de *Inicio*.
- “G”: casilla de *Meta*.

3.2. Mapeado del entorno a un MDP

En cuanto al espacio de estados, $\mathcal{S}$, está compuesto por todas las casillas que componen la rejilla, siendo que cada estado es único y está representado por sus coordenadas cartesianas en el formato $[y, x]$, donde el primer elemento sería un entero que representa la posición en el eje de ordenadas, cuyo sentido positivo es hacia abajo, y el segundo sería otro entero equivalente a la posición en el eje de abscisas, cuyos valores se incrementan hacia la derecha.

El entorno solamente está definido en la parte positiva de los dos ejes. De esta forma, el estado con los valores mínimos, el $[0, 0]$, será siempre el que se encuentre en la fila superior y en la columna más a la izquierda, mientras que el máximo sería el de la fila más baja y de la columna más a la derecha, cuya representación sería $[N_f, N_c]$. De esta forma, conociendo el número de filas y el número de columnas de la rejilla con la que se trabaje, podríamos acotar el espacio de estados como $(s \in \mathcal{S}) \forall [y, x] / (y \in [0, N_f]), (x \in [0, N_c])$, donde N_f y N_c serían valores enteros que se corresponden respectivamente con el número de filas y el de columnas de la rejilla.

El espacio de acciones $a \in \mathcal{A}$ lo conforman 4 elementos:

- **0 o Norte:** al realizar esta acción, un agente en el estado $s_t = [y, x]$ pasaría a estar en el estado $s_{t+1} = [y - 1, x]$
- **1 o Sur:** de $s_t = [y, x]$ se pasaría al $s_{t+1} = [y + 1, x]$
- **2 o Oeste:** de $s_t = [y, x]$ se pasaría al $s_{t+1} = [y, x - 1]$
- **3 o Este:** de $s_t = [y, x]$ se pasaría al $s_{t+1} = [y, x + 1]$

De esta forma, podríamos definir cada una de las transiciones $\mathcal{T}(s_{t+1}|s_t, a_t)$ realizables en el entorno (donde $s_t = [y_t, x_t]$ es el estado actual en el que se encuentra un agente, a_t la acción que realiza y $s_{t+1} = [y_{t+1}, x_{t+1}]$ el estado en el que acaba como resultado de realizar esa acción a_t estando en s_t) como una tupla de la forma $[y_t, x_t, a_t, y_{t+1}, x_{t+1}]$.

Sin embargo, es cierto que existen dos casos que son excepciones para esta norma, ya que cuando si se escoge una acción que suponga que el agente llegue a un estado que contiene un muro o que no está dentro de los rangos de $\mathcal{S}$, como s_{t+1} sería inválido, se decide que la transición se represente de la forma $[y_t, x_t, a_t, y_t, x_t]$, *i.e.* que el agente a pesar de haber realizado la acción, efectivamente no se haya movido del estado en el que ya estaba.

Además, es importante mencionar que se trata de un entorno determinista. El resultado de realizar la acción a_t en el estado s_t siempre será el mismo s_{t+1} , independientemente del momento t en el que suceda, por lo que no hay incertidumbre en las transiciones.

En cuanto a la función de recompensa, lo más adecuado para un entorno con un objetivo tan claro como es el de alcanzar la casilla de *Meta*, se decide que sea de 1 cuando el agente logre ese objetivo y de 0 en cualquier otro caso. Por tanto, teniendo esto en cuenta se puede definir definitivamente las tuplas que representan cada experiencia como $[y_t, x_t, a_t, y_{t+1}, x_{t+1}, r_{t+1}]$. Además, aparte de recibir la recompensa, al llegar al objetivo el episodio se terminaría.

3.3. Mapeado del MDP a un entorno OpenAI Gym

En esta sección se describen las particularidades de la implementación del MDP descrito en la sección 3.2 en un entorno *OpenAI Gym*. Como se describió en la sección 2.3, *OpenAI Gym* proporciona un *framework* de trabajo para la creación e interacción con dominios de aprendizaje por refuerzo. La creación de este entorno requiere la implementación de una función *step*, que va a ser la encargada, dada una acción y el estado en el que se encuentra el agente, de devolver cuál es el siguiente estado del agente y el refuerzo recibido en esa transición. Por lo tanto, esta función codifica la función de transición $\mathcal{T}(s_{t+1}|s_t, a_t)$ descrita en la sección 3.2. De esta forma, el agente puede interactuar de manera transparente con el entorno, sin necesidad de conocer los detalles de su implementación.

Cada vez que se termine un episodio, la función *reset* recoloca al agente en la posición de *Inicio* permitiendo volver a iniciar normalmente un nuevo episodio.

También se ha implementado la función *_is_valid_position* que, por seguridad, comprueba si al intentar realizar una acción desde *step* el agente termina en una casilla correcta o si por contra se sale de los límites del entorno o acaba en una casilla que contiene un obstáculo.

Con el objetivo de conseguir mayor variación en los experimentos, se permite definir más de una casilla inicial. Por este motivo, se implementa la función *randomize_start_pos*, que permite elegir una nueva casilla de *Inicio* aleatoriamente de entre las definidas como posibles.

Por último, se implementa la función *render* que, ayudándose de la librería *Pygame*, es la que permite la visualización gráfica del entorno como podemos ver en las figuras 3.1 y 3.2, haciendo uso del código de colores ya explicado en la sección 3.1. El renderizado del entorno no es condición indispensable para la ejecución de los experimentos y, de hecho, se recomienda no usarlo a no ser que sea totalmente necesario porque ralentiza notablemente la ejecución del código. Con este fin, puede ser activado o desactivado en función del valor que le demos al *booleano render*, que es uno de los parámetros iniciales del entorno junto con el mapa de caracteres.

4 Aprendizaje por refuerzo *offline* basado en el modelo con gestión de incertidumbre

En este capítulo, se describe la estrategia desarrollada para el aprendizaje de políticas de comportamiento de forma *offline* a partir de un conjunto de datos previamente recopilados. Para lograrlo, se ha diseñado un *ensemble* de redes neuronales con el que es posible medir la incertidumbre asociada a un estado s_t concreto, para así poder diferenciar entre los estados presentes en el dataset y los que no. Con ello se consigue un método efectivo para resolver el problema del *distributional shift*, descrito previamente en la sección 2.2.2.

4.1. Generación del dataset *offline*

En un primer momento, el enfoque propuesto para obtener las experiencias para la generación del dataset de entrenamiento para el modelo *offline* consistía en el diseño de agentes demostradores. Estos demostradores, tienen como objetivo el aprender una política π a base de interactuar con el entorno, para posteriormente, haciendo uso de esa política, generar trayectorias que contengan experiencias válidas para añadir al dataset. Para ello, usando como base los códigos presentes en [28] y [29], se implementan 3 clases de agentes: *QLearning*, *DQN* y *DDQN*.

De esta forma, entrenando un buen número de agentes y usando distintas casillas de inicio para cada uno, se puede obtener una cantidad de trayectorias que contengan suficientes experiencias como para generar el dataset de entrenamiento. Todas aquellas experiencias no incluidas en él por no estar presentes en la trayectoria son las que se consideran fuera de la distribución de datos del dataset, aquellas que se deben evitar a la hora de entrenar agentes a partir del dataset por causa del ya conocido *distributional shift*.

Sin embargo, con este entorno no resulta estrictamente necesario el uso de ningún tipo de agente demostrador. Existe una opción mucho más directa y cómoda de implementar, que consiste en la generación de datos sintéticos para el dataset. Como se tiene acceso a la rejilla del entorno en su formato de arrays de caracteres y se conocen las acciones que componen $\mathcal{A}$, es muy sencillo simular todas las experiencias posibles en el entorno. Para simular el *distributional drift*, se define un nuevo carácter posible para los estados de la rejilla: aquellas posiciones con una “x” se consideran como no demostradas, *i.e.* que el dataset no incluirá experiencias en las que un estado “x” sea casilla de origen o destino.

A la hora de personalizar la configuración del laberinto, debe tenerse cuidado de que se deje al menos una ruta posible de casillas demostradas entre la casilla de *Inicio* y la *Meta*, ya que si no el entorno resultante será irresoluble por los agentes, ya que, como se explicará más a fondo en la sección 4.4, serán penalizados cada vez que visiten una de esas casillas, con el fin de evitar el problema del *distributional shift*.

Con el fin de generar el dataset sintético se implementa el Algoritmo 2.

Algoritmo 2: Generador de Dataset Sintético

Input: *grid*

Output: *train_experiences*

train_experiences $\leftarrow$ [...]

n_rows, n_cols $\leftarrow$ dimensiones del *grid*

for *y* *in range* [0, *n_rows*] **do**

for *x* *in range* [0, *n_cols*] **do**

for *a* *in range* [0, 3] **do**

if *a* == 0 **then**

 // Norte

$y1, x1 \leftarrow \max(0, y - 1), x$

else if *a* == 1 **then**

 // Sur

$y1, x1 \leftarrow \min(n_rows - 1, y + 1), x$

else if *a* == 2 **then**

 // Oeste

$y1, x1 \leftarrow y, \max(0, x - 1)$

else if *a* == 3 **then**

 // Este

$y1, x1 \leftarrow y, \min(n_cols - 1, x + 1)$

if *maze*[*y, x*] == "." **then**

 // Si el origen es una casilla libre y demostrada

if *maze*[*y1, x1*] == "x" **then**

 // Si el destino es una casilla no demostrada

pass

else if *maze*[*y1, x1*] == "#" **then**

 // Si el destino es un muro

train_experiences.append([*y, x, a, y1, x1*])

else

 // Si destino es una casilla libre y demostrada

train_experiences.append([*y, x, a, y1, x1*])

return *train_experiences*

Aparte de no incluir las casillas no demostradas, tampoco se tienen en cuenta las transiciones donde el origen sea una casilla con un muro. Además, las experiencias tienen el formato $[y_t, x_t, a_t, y_{t+1}, x_{t+1}]$, excepto cuando la casilla de destino no sea un

muro o se salga de los límites físicos del *grid*, en cuyo caso será $[y_t, x_t, a_t, y_t, x_t]$ como ya se había explicado en 3.

4.2. Construcción del *ensemble* de modelos

La propuesta que se presenta en este proyecto es la de, haciendo uso de los datos sintéticos generados según lo expuesto en la sección 4.1, implementar de forma *offline* un modelo del entorno en forma de *ensemble* de redes neuronales. Este *ensemble* se compone de redes de neuronas m_i con $i \in \{1, \dots, N\}$, todas entrenadas a partir del mismo conjunto de datos, pero con diferentes vectores de parámetros iniciales θ_i , introduciendo cierta variación con el fin de que los modelos generados no sean idénticos. De esta manera, cada m_i en el *ensemble* codifica una función $\mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ que permite mapear pares estado, acción, a un estado siguiente.

El objetivo es lograr que las predicciones de las redes neuronales m_i sean próximas entre ellas en aquellas transiciones que se encuentren dentro del dataset de entrenamiento, pero presenten una mayor variación para las transiciones que no se encuentren en la distribución de datos con la que se han entrenado. Lo que se busca con esto es lograr la implementación de un mecanismo de control que, mediante el análisis de la desviación estándar (*stdD*) de las predicciones de todos los modelos, siguiendo el proceso que se describe en la siguiente sección 4.3, permita detectar cuando un estado está afectado por el problema del *distributional shift* para que pueda ser evitado por los agentes.

Para la implementación de los modelos se emplea la librería *PyTorch* [30]. Las redes neuronales reciben como entrada un tensor¹ con 3 elementos con la información del estado s_t y la acción a_t , con la forma $[y_t, x_t, a_t]$, y a su salida devolverá como resultado el tensor $[y_{t+1}, x_{t+1}]$, con la información del estado siguiente s_{t+1} .

Pongamos por ejemplo la transición $[0, 0, 1, 1, 0]$, donde el estado inicial $s_t = [0, 0]$, la acción es 1 o *Sur* y el estado siguiente $s_{t+1} = [1, 0]$. Si se entrenase a los modelos con un dataset que incluye esta transición, al pasarles como tensor de entrada $[0, 0, 1]$, los resultados obtenidos serían $[0.89, 0.14]$, $[1.03, 0.06]$ o $[0.95, 0.2]$, todos valores que, aunque distintos, se aproximan mucho al valor real $[1, 0]$ y, lo más importante de todo, muy próximos entre ellos. Si, por contra, esa transición no estuviese presente en el dataset de entrenamiento de las redes, los resultados obtenidos serían tensores como $[2.63, 5.76]$, $[0.03, 1.49]$ o $[3.32, 0.33]$, que son resultados muy dispares, por lo que al calcular la desviación estándar existente entre ellos se obtendría un valor considerablemente mayor que en el caso anterior.

De esta manera, aunque no se pueda saber *a priori* si los valores de las predicciones se corresponden con las coordenadas reales en el entorno, si que se puede extraer esa información de manera indirecta al aplicar esta metodología.

Sin embargo, como se adelantaba en la sección 3.1, las características propias de este entorno (que tanto el espacio de estados como el de acciones sean discretos), supone complicaciones extra cuando se juntan con la alta capacidad para la generalización

¹En *PyTorch*, un tensor es una estructura de datos multidimensional que representa datos numéricos y es fundamental para realizar operaciones con redes neuronales. Puede ser un escalar (0D), un vector (1D), una matriz (2D) u otras dimensiones superiores.

que presentan las redes neuronales. Es muy fácil que en lugar de aprender las transiciones específicas $\mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ presentes en el dataset, las redes acaben imbuyendo la función que define el movimiento en el entorno, *i.e.* que cuando la acción sea *Sur* hay que restar 1 a la coordenada y , cuando sea *Este* restar 1 a la x , etc.

Este tipo de comportamiento podría llegar a ser positivo en otros casos, pero catastrófica en otros: puede que la generalización prevea que una transición no lleva a un obstáculo cuando realmente sí lo hace. Esto no es un problema de demasiado calibre en un dominio simulado, pero cuando nos encontramos en un entorno real con situaciones catastróficas, esta generalización podría conducir a daños irreparables. Por lo tanto, en este proyecto se limita que la generalización se lleve a cabo en zonas muy próximas a casillas demostradas, siendo esta menor en cuanto comenzamos a alejarnos de ellas.

4.3. Cálculo de la incertidumbre

La idea propuesta para el cálculo de la incertidumbre, dado un estado s con coordenadas $[y, x]$ y las 4 posibles acciones $[Norte, Sur, Oeste, Este]$, consiste en lo siguiente:

1. Para una acción a posible en s , se crea un tensor $[y, x, a]$. Con este tensor se calculan las predicciones p_i con cada uno de los m_i modelos en el *ensemble*, de la forma $p_i = [y, x]$ con $i \in \{1, \dots, N\}$.
2. Se calcula la desviación estándar $stdD$ de las predicciones:

$$stdD_a = \sqrt{\frac{1}{N} \sum_{i=1}^N (p_i - \bar{p})^2} \quad (4.1)$$

donde N es el número de modelos, $\bar{p}$ es la media de las predicciones para cada acción, y p_i es la predicción i -ésima dentro del *ensemble* del modelos.

3. Como es un vector de 2 elementos, se calcula el módulo de la desviación estándar para facilitar los cálculos:

$$mod_stdD_a = |stdD_a| \quad (4.2)$$

4. Se repiten todos los pasos anteriores con las otras 3 acciones restantes. Una vez se tienen los 4 resultados se calcula su media, que será la incertidumbre asociada al estado s :

$$incertidumbre_s = \frac{1}{4} \sum_{a=0}^3 mod_stdD_a \quad (4.3)$$

El pseudocódigo con la implementación del cálculo de la incertidumbre se puede ver en el Algoritmo 3:

Una vez calculada la incertidumbre para todos los estados, se obtiene un modelo en el que, tras encontrar un valor adecuado para el umbral, representado como ρ , ya es posible diferenciar aquellos estados presentes en el dataset *offline* de los que no.

Algoritmo 3: Calcular Incertidumbre de un Estado

Input: Estado: $s = [y, x]$
 Acciones: $A = [0, 1, 2, 3]$
 Modelos: $M = [M_1, M_2, \dots, M_i]$

Output: Incertidumbre asociada a s : $incertidumbre_s$

```

arr_predictions  $\leftarrow$  [...]
arr_stdD  $\leftarrow$  [...]

// Para cada acción:
for  $a \in A$  do
    // Se hace una predicción con cada modelo del ensemble
    for  $M_i \in M$  do
        prediction  $\leftarrow$   $M_i.predict([y, x, a])$ 
        arr_predictions.append(prediction)

    // Se calcula el módulo de la stdD asociada a la acción  $a$ 
    stdD_a  $\leftarrow$  mod(stdD(arr_predictions))
    arr_stdD.append(stdD_a)

// Se calcula  $incertidumbre\_s$  como la media de stdD de las 4 acciones
incertidumbre_s  $\leftarrow$   $\frac{1}{4} \sum_{a=0}^3 arr\_stdD_a$ 

```

En las figuras 4.1 y 4.2 se puede observar una correspondencia del 100 % entre las casillas reales no demostradas y las predichas por el modelo.

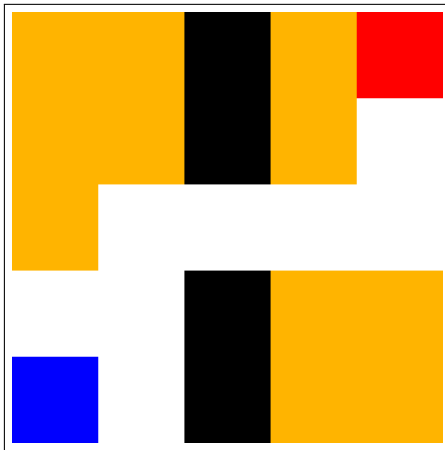


FIGURA 4.1: En naranja, casillas no demostradas

0.953	0.843		0.392	0.137
0.707	0.508		0.391	0.119
0.384	0.126	0.005	0.102	0.084
0.075	0.004		0.473	0.497
0.004	0.006		0.643	0.782

FIGURA 4.2: En rojo, casillas con una incertidumbre superior a un umbral $\rho = 0.3$

En la figura de la izquierda se puede una configuración sencilla para un laberinto de 5x5, con la casilla de *Inicio* en verde (en este caso está situada en la misma casilla que la posición actual del agente, que tiene preferencia de renderizado), la casilla de *Meta* en rojo, los obstáculos en negro y, lo más importante de todo, con una diferenciación entre las casillas libres demostradas y no demostradas. Aquellas casillas libres que forman parte del dataset de entrenamiento del modelo se muestran de color blanco (además de la *Meta* y el *Inicio*) y las que no en color naranja. En la figura

de la derecha se muestra, para la misma configuración de *grid*, el resultado de calcular la incertidumbre de cada casilla haciendo uso del Algoritmo 3 y del *ensemble* de modelos. Después de fijar el umbral $\rho = 0.3$, se colorean de rojo los valores que lo superen o igualen y en verde los que sean inferiores. De esta manera, se puede ver como existe una correspondencia total entre los estados que estaban pintados de naranja en la figura de la izquierda y los que superan el umbral de incertidumbre en esta.

4.4. Introducción de la incertidumbre en la función de refuerzo

Para lograr entrenar agentes que, haciendo uso de los modelos obtenidos, sean capaces de evitar aquellos estados no demostrados, evitando eficazmente caer en el problema del *distributional shift*, basta con modificar la función de recompensa. La estrategia más efectiva consiste en penalizarlos, *i.e.*, darles una recompensa negativa, cada vez que durante su entrenamiento lleguen a alguno de los estados con una incertidumbre superior al umbral ρ que se considere.

Así, de la misma manera que la recompensa positiva por llegar a la *Meta* se va retropropagando a través de los estados inmediatamente anteriores para “guiar” al agente hacia ella, la penalización lo hará de forma inversa, forzando a que los agentes eviten pasar por esa casilla no demostrada. En algunas ocasiones, cuando hay demasiadas casillas no demostradas juntas, podría llegar a suponer que el agente sea incapaz de seguir la ruta más “natural” hacia la *Meta*, teniendo que explorar nuevas alternativas.

De esta forma, la definición formal de la nueva función de recompensa r_{new} sería:

$$r_{new} = r - r_{incertidumbre} \quad (4.4)$$

donde r sería la función de refuerzo extrínseca inherente al entorno, mientras que $r_{incertidumbre}$ es la función de recompensa que penaliza la incertidumbre asociada a un estado.

La fórmula específica para calcular $r_{incertidumbre}$ puede variar según la naturaleza del entorno y la información disponible. La idea básica, que es la que se implementa en este proyecto, consiste en emplear una función escalón para penalizar con un valor v_{pen} fijo la visita a cualquier estado cuya incertidumbre supere el umbral de incertidumbre ρ . Esta función sería tal que:

$$r_{incertidumbre} = \begin{cases} v_{pen} & \text{si } incertidumbre_s \geq \rho \\ 0 & \text{si } incertidumbre_s < \rho \end{cases} \quad (4.5)$$

donde $incertidumbre_s$ era el valor de incertidumbre asociado al estado s calculado en la Ecuación 4.3.

Sin embargo, también existe la posibilidad de realizar el cálculo de $r_{incertidumbre}$ para cualquier estado, usando el valor de incertidumbre del mismo para ponderar el valor de la penalización con la siguiente fórmula:

$$r_{incertidumbre} = incertidumbre_s * v_{pen} \quad (4.6)$$

donde, cuanto mayor sea la incertidumbre asociada al estado, mayor será también la penalización $r_{incertidumbre}$ recibida por visitarlo.

5 Resultados

En este capítulo se presentan los resultados experimentales del proyecto. En primer lugar, se expone un experimento realizado para escoger el mejor método para la creación del dataset *offline* de entre tres candidatos posibles. Después, se muestran como, para 4 configuraciones distintas del entorno, se entrenan sendos *ensembles* para modelar esos entornos concretos, lo que posteriormente permite entrenar agentes que sean capaces de encontrar un camino entre la casilla de *Inicio* y la de *Meta* logrando evitar efectivamente los estados no demostrados.

Todos estos resultados se corresponden con el segundo y definitivo enfoque propuesto en este proyecto (generación del dataset con transiciones sintéticas), mientras que los resultados que se obtuvieron durante el primero (uso de agentes demostradores para generar las experiencias) se pueden encontrar en el Apéndice B.

5.1. Experimentos de generación del dataset

Como se explica en el Capítulo 4, para generar el dataset con el que se entrenarán los modelos se emplea el Algoritmo 2. Con el objetivo de poder obtener una métrica para controlar el grado de sobreajuste de las redes durante el entrenamiento, todas aquellas transiciones que son posibles en el entorno pero que no se agreguen al dataset de entrenamiento se añaden al dataset de validación. De esta forma se puede ver como evoluciona el grado de generalización de las redes durante todas las épocas que entrenen.

En principio todas las redes son entrenadas usando el mismo conjunto de datos, compuesto por el 100 % de las transiciones presentes en el dataset *offline*, pero según la teoría [31] debería existir cierta variación entre los datos de entrenamiento de cada una de las redes, para así conseguir modelos ligeramente diferentes en el *ensemble*. Por ello, se diseña un experimento en el que se entrenan 3 *ensembles* distintos para el mismo *grid* (de 14×14), cada uno usando una estrategia distinta para la configuración de sus dataset de entrenamiento:

- **Modo 1:** El método que se usaba previamente. Todas las redes se entrenan usando el conjunto de datos al completo, tal cual como se genera con el Algoritmo 2. Todas las redes tienen el mismo conjunto de datos tanto de entrenamiento como de validación.
- **Modo 2:** Para cada modelo, se escoge aleatoriamente un 33 % de las transiciones del dataset de entrenamiento, y se duplican. De esta forma, cada uno tiene un dataset único de entrenamiento, pero el de validación será el mismo para todos.
- **Modo 3:** Para cada modelo, se seleccionan aleatoriamente un 33 % de las transiciones del dataset de entrenamiento y se agregan a su dataset de validación. De

las transiciones que quedan en el conjunto de datos de entrenamiento, selecciono aleatoriamente un 33 % para su duplicado. Así, cada modelo tiene unos dataset de entrenamiento y validación únicos.

Después de entrenar un *ensemble* para cada modo siguiendo los métodos expuestos, se obtienen los siguientes resultados:



FIGURA 5.1: En (A) se puede ver la configuración de mapa elegida para este experimento con los estados no demostrados en naranja y en (B), (C) y (D) la incertidumbre para cada estado para los 3 modos a comparar

Ante los resultados presentados se puede observar claramente como con el *Modo 3* resulta imposible diferenciar sobre el *grid* aquellos estados que están fuera o dentro de la distribución de entrenamiento según su incertidumbre, incluso a pesar de haber aumentado el umbral ρ a 0.6, ya que la incertidumbre es bastante alta para todos los estados en general. Es importante destacar que empleando el mismo conjunto de entrenamiento para cada red del *ensemble* se obtienen unos resultados muy

positivos. Esto nos indica que la variación introducida en los modelos por una inicialización de los pesos de cada red diferente, resulta suficiente para incluir variedad en el *ensemble* que después permite distinguir claramente los estados para los que no existen demostraciones.

En cuanto a los *Modo 1* y *Modo 2*, el resultado obtenido es bastante similar, aunque si se hace una comparación estricta, se puede ver como el primero de ellos, aunque por poco, obtiene mejores resultados. Ante los datos resultados de este experimento, se decide conservar el método *Modo 1* para la creación del resto de *ensembles* del proyecto, aunque el uso de cualquiera de los dos podría ser válido tras realizar un buen ajuste del umbral.

5.2. Creación de ensembles offline

En esta sección, se presentan los resultados finales del proyecto. Para cada uno de los dos tamaños de *grid*, se presentan 2 configuraciones distintas cambiando la localización de las casillas no demostradas. Después, se muestra para cada una la incertidumbre de cada estado dada por su *ensemble* correspondiente. Por último, se puede ver como agentes encuentran con éxito una ruta del *Inicio* a la *Meta*, evitando aquellas casillas que no fuesen demostradas y por tanto el problema del *distributional drift*.

Para cada una de las diferentes configuraciones de entornos que se creen se debe entrenar un *ensemble* distinto. Esto se debe a que cada mapa, al tener distintos tamaños o distribución de casillas no demostradas que el resto, genera un dataset único y los *ensembles* solo son capaces de modelar el dataset con el que son entrenados.

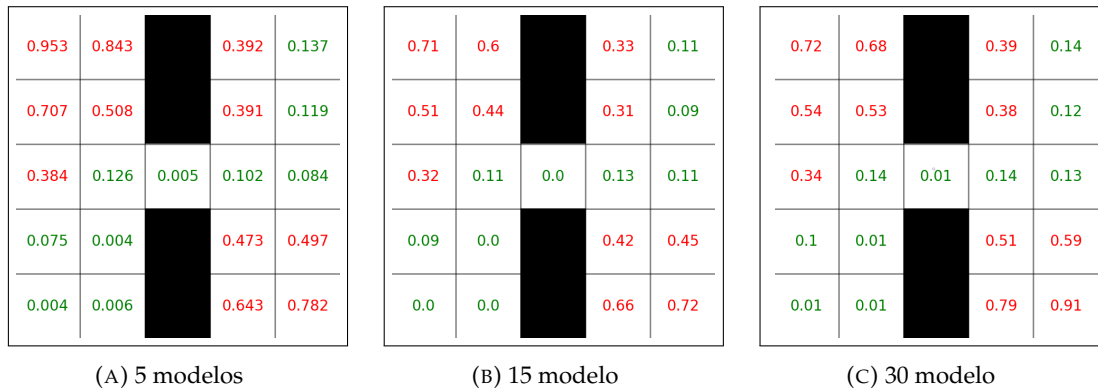


FIGURA 5.2: Comparativa del rendimiento en el cálculo de la incertidumbre con distinto número de modelos en el *ensemble* para el entorno 5x5

Respecto al número de redes que componen cada *ensemble*, se ha decidido optar por 5 para todos los casos, independientemente del entorno que se use. En teoría, cuantos más modelos más fácil sería realizar la tarea ya que, mientras que la incertidumbre se mantendría muy baja para los estados demostrados, debería crecer para los no demostrados a medida que lo haga también el número de modelos y más fácil sería hacer la distinción. Sin embargo, al aumentar la cantidad de modelos también se aumentan el tiempo y el coste computacional para entrenarlos, así como también para la fase de inferencia. Por ello, se ha encontrado experimentalmente que 5 es el

mejor número de modelos para los *ensembles* ya que permite diferenciar, junto con un umbral $\rho = 0.3$ para la incertidumbre, los estados de los no demostrados casi en el 100 % de los casos, como se demostrará en esta sección de la presente memoria. Además, como se puede observar en la Figura 5.2 se han entrenado *ensembles* de 5, 15 y 30 modelos para el entorno de 5x5 (aunque estos resultados son extrapolables al 14x14) para comparar las incertidumbres asociadas a cada estado que calcula cada uno de ellos, sin obtener resultados que mejoren sustancialmente los actuales.

En cuanto a la implementación de las redes que componen los *ensembles*, se usa la librería *PyTorch*. Estas van a recibir como entrada un tensor con 3 elementos con la información del estado s_t y la acción a_t , con la forma $[y_t, x_t, a_t]$, y a su salida devolverá el tensor $[y_{t+1}, x_{t+1}]$, con la información del estado siguiente s_{t+1} , como se indicaba en el Algoritmo 3.

Todas las capas que componen las redes usan activación ReLU. En cuanto a su arquitectura, se podemos diferenciar a las redes entrenadas para el entorno de 5x5 de las de 14x14 (entre paréntesis el número de neuronas en esa capa):

- **5x5:** $Input(3) \rightarrow HiddenLayer(32) \rightarrow HiddenLayer(32) \rightarrow Output(2)$
- **14x14:** $Input(3) \rightarrow HiddenLayer(512) \rightarrow HiddenLayer(512) \rightarrow Output(2)$

Para la función de recompensa se usan las ecuaciones 4.4 y 4.5. Los valores de las variables r y v_{pen} elegidos dependen del tamaño del entorno, su distribución de casillas no demostradas y del tipo de agente. Con el fin de facilitar la reproducibilidad de estas pruebas, se ha decidido usar agentes *QLearning*, que buscan la política óptima mediante el Algoritmo 1, ya que los *DQN* y *DDQN*, aunque también son capaces de resolver la tarea propuesta, presentan mayor variabilidad de resultados de una ejecución a otra incluso sin cambiar el entorno, con lo que el ajuste paramétrico resulta algo más complejo. Para el tipo de agentes elegido, se encuentra experimentalmente que con los valores $r = 1000$ y $v_{pen} = 100$ se consiguen generar políticas acertadas para cualquiera de los entornos en los que se experimenta.

En las siguientes secciones, se muestran gráficamente y se analizan los resultados obtenidos para las cuatro configuraciones de entorno presentadas.

5.2.1. Grid de 5x5: distribución 1

En esta sección se explican los resultados de la primera configuración de *grid* que se muestra en la Figura 5.3 (A), usando un número máximo de 1000 episodios y con un máximo de 100 pasos por episodio. En esta figura se distinguen como siempre la casilla de inicio que se corresponde en la que se encuentra el agente (azul), la casilla meta (roja), y los obstáculos (negro). También en naranja se incluyen las casillas para las cuales se ha decidido no incorporar ninguna demostración. Es esperable, por tanto, que sean estas casillas las que más incertidumbre acumulen después.

Esto se puede comprobar en las Figuras 5.3 (B) y (C) donde se muestra la incertidumbre calculada por cada acción dentro de un estado, y la acumulada por cada estado, respectivamente. En este caso, se puede comprobar que se hace un mapeo perfecto entre las casillas naranjas y las consideradas con una alta incertidumbre. Cabe destacar también algunas transiciones que se consideran que no tienen incertidumbre cuando realmente la tienen porque no han sido demostradas. Esto ocurre por ejemplo, en los estados $[2, 0]$ con la acción de moverse a la izquierda, y el estado

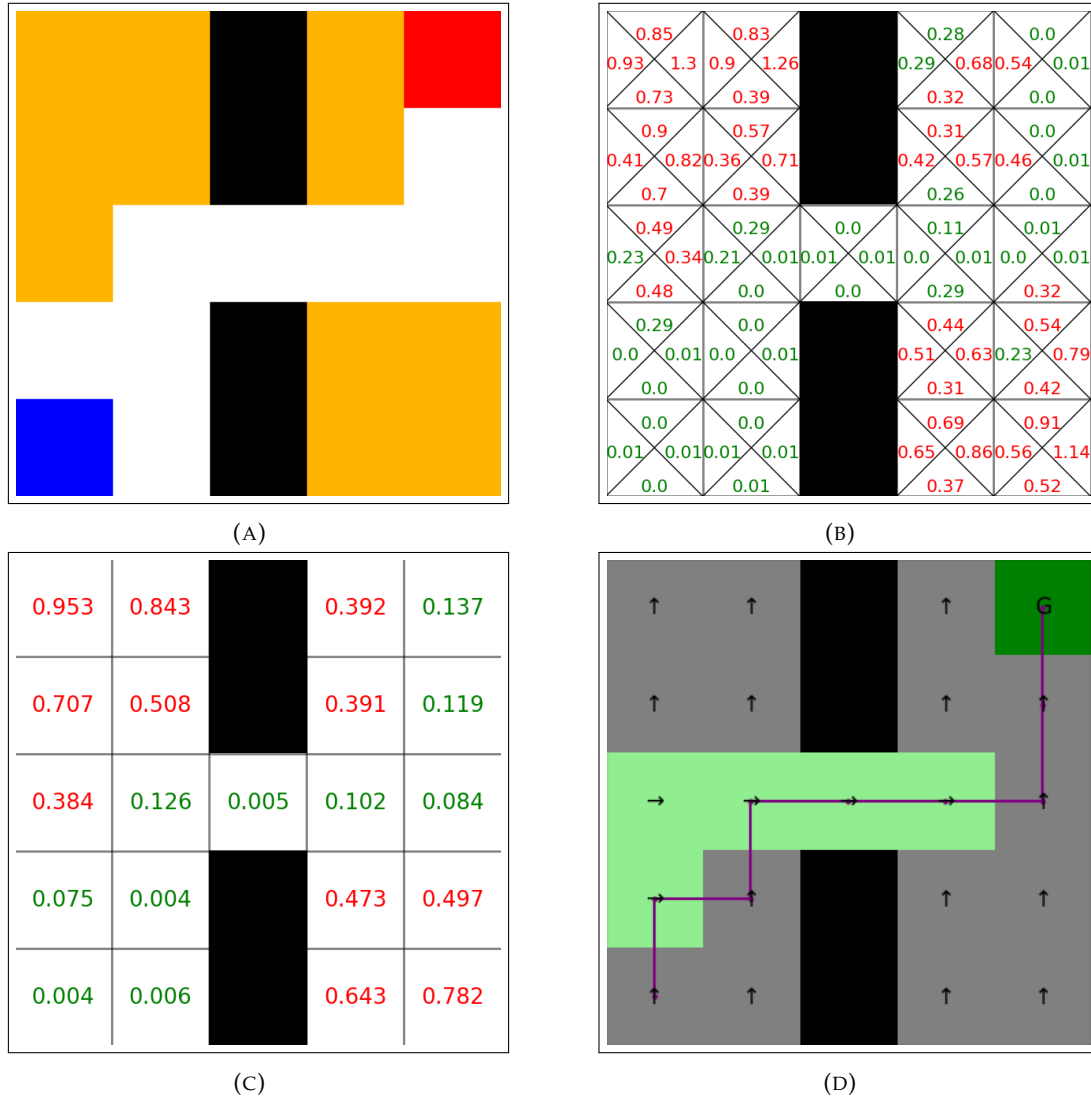


FIGURA 5.3: En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el *ensemble* asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este *ensemble*, que evita correctamente los estados no demostrados.

[0,3] con las acciones de moverse arriba y a la izquierda. Esto se debe a los efectos de la generalización y se nota más en estados que se encuentran en los límites entre regiones demostradas y no demostradas. En cualquier caso, gracias a la incertidumbre acumulada, y estableciendo un valor para ρ correcto, es posible distinguir estos estados también como estados con incertidumbre.

Finalmente, en la Figura 5.3 (D) se muestra la política aprendida cuando se emplea el algoritmo 1 y la función de refuerzo de la ecuación 4.5. Se puede observar claramente como la ruta seguida evita casillas con una alta incertidumbre. En la Figura 5.4 se puede ver la evolución de este proceso de aprendizaje. En particular, se muestra la evolución del refuerzo obtenido (gráfica más a la izquierda), la evolución de la longitud de los episodios (gráfica central) y por último la evolución del error de entrenamiento (gráfica de la derecha). Se puede observar que el aprendizaje de la política de comportamiento se hace muy rápido, obteniendo la convergencia alrededor del

episodio 150. Esta convergencia se puede analizar también considerando el número de pasos: a medida que el algoritmo aprende menor será el número de pasos que necesita para llegar a la casilla de destino. Por último, en cuanto al error que se muestra en la gráfica de la derecha, se trata de las diferencias temporales, que indican la magnitud total de la actualización de la tabla Q que emplea el agente $QLearning$ para la búsqueda de la política más adecuada. Gracias a esta gráfica, se puede ver como al principio del experimento no se producen actualizaciones considerables de los valores de la tabla, pero tras alcanzarse la meta por primera vez empieza a retropropagarse la recompensa, comenzando así el proceso de actualización, por lo que el valor de las diferencias temporales aumenta. Luego se va atenuando hasta que la tabla comienza a converger a la política definitiva. Además, la magnitud de actualización puede ser tanto positiva como negativa, dependiendo de en que sentido se actualicen los valores de la tabla. Por último, señalar que la diferencia de escala en el eje X entre esta gráfica y las anteriores se debe a que se realiza una actualización por cada paso, por lo que el número total de actualizaciones sería el resultado de sumar los pasos totales de todos los episodios.

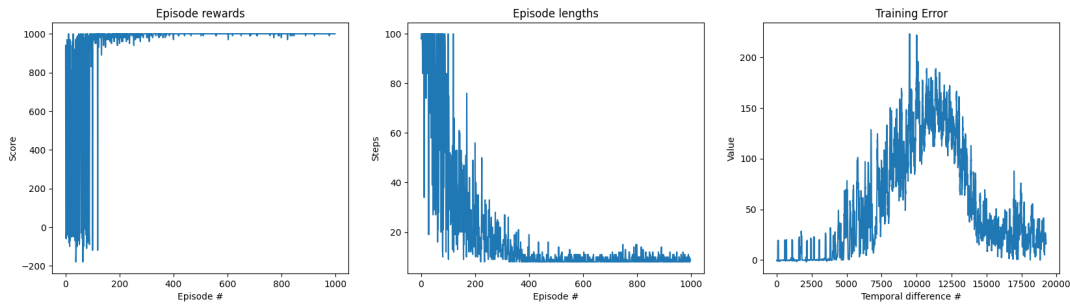


FIGURA 5.4: Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)

5.2.2. Grid de 5x5: distribución 2

A continuación, en este segundo experimento, se continuará con el mismo tamaño de *grid* que se utilizó en la sección anterior y con el mismo número máximo de episodios y de pasos por episodio, pero la distribución de las casillas no demostradas será diferente para comprobar que efectivamente, el método planteado es capaz también de detectar estas casillas de una alta incertidumbre. De manera similar a la anterior, en la Figura 5.5 (A) se muestra la configuración de este *grid*, y en las Figuras 5.5 (B) y (C) se muestra la incertidumbre calculada por transición y por estado, respectivamente. En la Figura 5.5 (D) se muestra la política aprendida por el agente $QLearning$.

En la Figura 5.6 se pueden observar las estadísticas del proceso de aprendizaje del agente, bastante similares a las obtenidas en el anterior experimento. Sin embargo, esta vez se observa dos fenómenos:

- La meta se encuentra por primera vez poco después de comenzar el experimento, lo que explica que la gráfica del error de entrenamiento también crezca más temprano que antes.
- Aunque la meta se encuentre antes y la longitud de los episodios disminuye rápidamente, hay bastantes episodios en los que la recompensa se queda algo

por debajo del máximo. Esto se puede deber a que todavía se sigue explorando o a que la política todavía no está optimizada del todo, por lo que aún se están visitando algunos estados que penalizan.

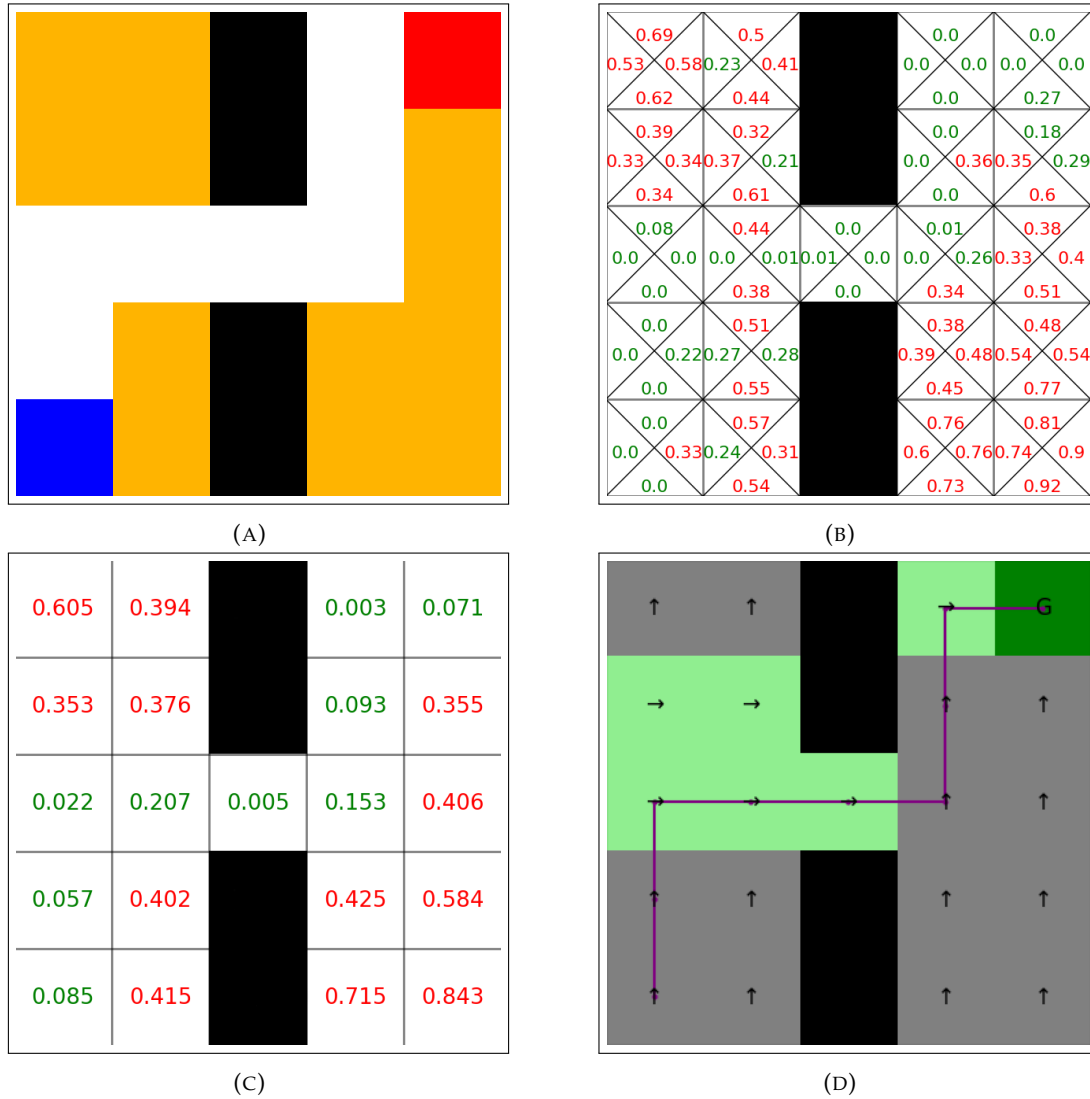


FIGURA 5.5: En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el *ensemble* asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este *ensemble*, que evita correctamente los estados no demostrados.

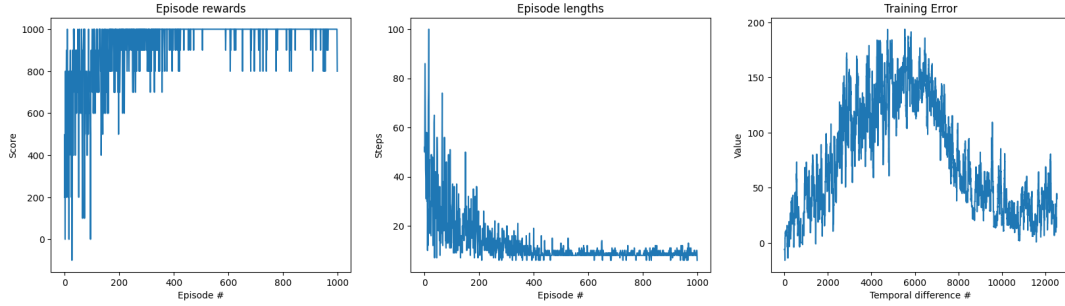


FIGURA 5.6: Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)

5.2.3. Grid de 14x14: distribución 1

En esta sección se explican los resultados de la primera configuración de *grid* de tamaño 14x14 que se muestra en la Figura 5.7 (A). Para adecuarse al tamaño y complejidad del *grid*, se ha aumentado el número máximo de episodios a 3000 y el número máximo de pasos por episodio a 300.

En las Figuras 5.7 (B) y (C), donde se muestra la incertidumbre calculada por cada acción dentro de un estado, y la acumulada por cada estado, se puede comprobar que se hace un mapeo casi perfecto entre las casillas naranjas y las consideradas con una alta incertidumbre. Sin embargo, en las casillas [7, 0] y [8, 0], a pesar de existir 3 acciones con una incertidumbre casi nula para 3 de las 4 acciones, la magnitud de la incertidumbre de la acción *Este* es tan grande que al calcular la incertidumbre total del estado, el resultado es superior al umbral ρ , clasificándolas incorrectamente. Aún así, este error no es de mayor importancia, ya que estas casillas están aisladas totalmente del resto de estados demostrados, lo cual a su vez es parte del motivo de que su incertidumbre sea así de alta.

En la Figura 5.7 (D) se muestra la política aprendida por el agente *QLearning*, en la que podemos comprobar como la trayectoria aprendida evita con éxito las casillas con una alta incertidumbre, mientras que en la Figura 5.8 se pueden ver las estadísticas obtenidas durante el proceso de aprendizaje, que como en los experimentos anteriores muestra la evolución del refuerzo obtenido, la evolución de la longitud de los episodios y la evolución del error de entrenamiento. Se puede observar que el aprendizaje de la política de comportamiento se hace muy rápido, obteniendo la convergencia alrededor del episodio 500, que es también cuando el número de pasos por episodio comienza a caer con rapidez.

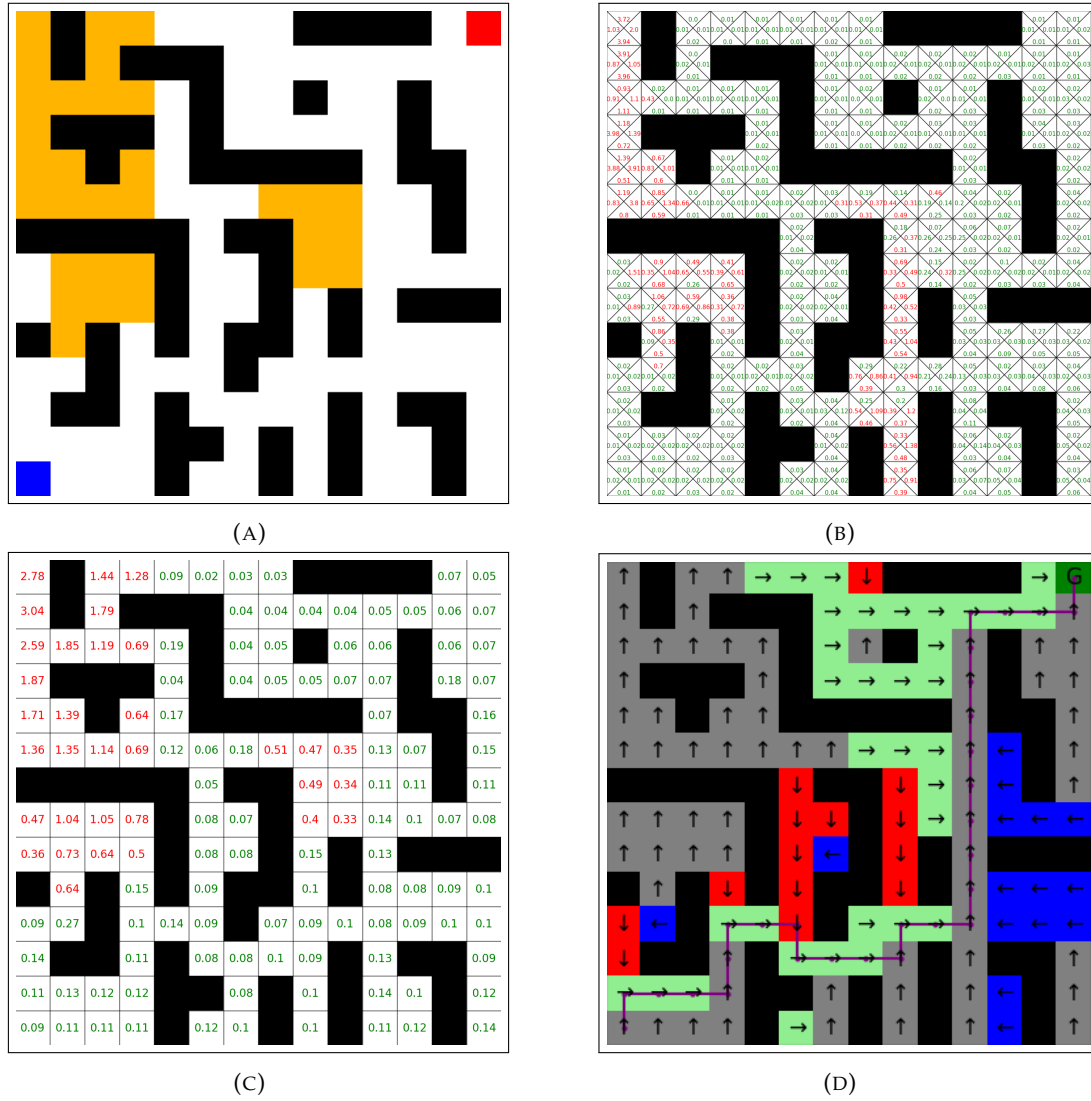


FIGURA 5.7: En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el *ensemble* asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este *ensemble*, que evita correctamente los estados no demostrados.

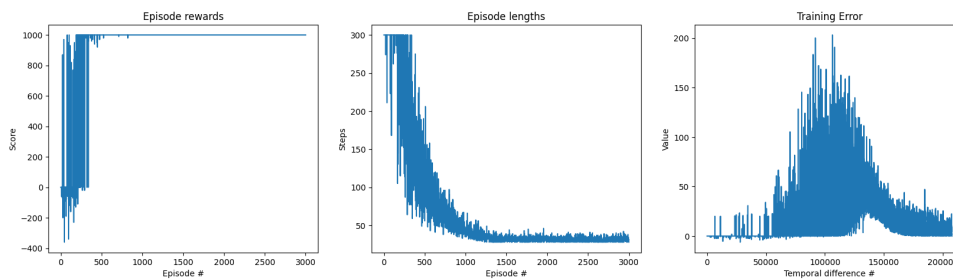


FIGURA 5.8: Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)

5.2.4. Grid de 14x14: distribución 2

En este segundo experimento con el entorno de 14x14 se mantiene el mismo número máximo de episodios y de pasos por episodio que en el anterior, pero se varía la distribución de las casillas no demostradas para demostrar nuevamente que el método planteado es capaz de funcionar correctamente. Como en los otros experimentos, en la Figura 5.9 (A) se encuentra la configuración del *grid* del entorno empleado, en las Figuras 5.9 (B) y (C) se muestra la incertidumbre calculada para cada transición y para cada estado respectivamente, y en la Figura 5.9 (D) se muestra la política aprendida por el agente *QLearning*.

Por último, en la Figura 5.10 se pueden observar las estadísticas del proceso de aprendizaje del agente. En este experimento resulta que hay algunos episodios en los que se acumula una recompensa total de valores incluso menores que -25000 . Esto se debe a la columna de casillas no demostradas que se encuentran en la parte central del mapa. Como el camino más corto hacia la meta pasa por ellas y la navegación del agente depende de las predicciones $[y_{t+1}, x_{t+1}]$ que calcula el *ensemble*, es muy fácil que llegue a una casilla no demostrada de esa zona, y que al hacerlo se quede atrapado en ella, acumulando una cantidad ingente de penalizaciones, lo que explica los valores de la gráfica. Sin embargo, esto no evita que, una vez se alcance la meta, la retropropagación de la recompensa correspondiente permita al agente descubrir exitosamente la política óptima, como sucedía en los entornos de los experimentos anteriores.

En cuanto a la gráfica de error de entrenamiento (gráfica más a la derecha en la Figura 5.10), destacar que es en la que mejor se aprecian actualizaciones en sentido negativo. Este fenómeno se debe a que durante el inicio del experimento se estaba convergiendo hacia una política que no era correcta, y al descubrirse la meta y comienza a corregirse, generando las diferencias temporales negativas.

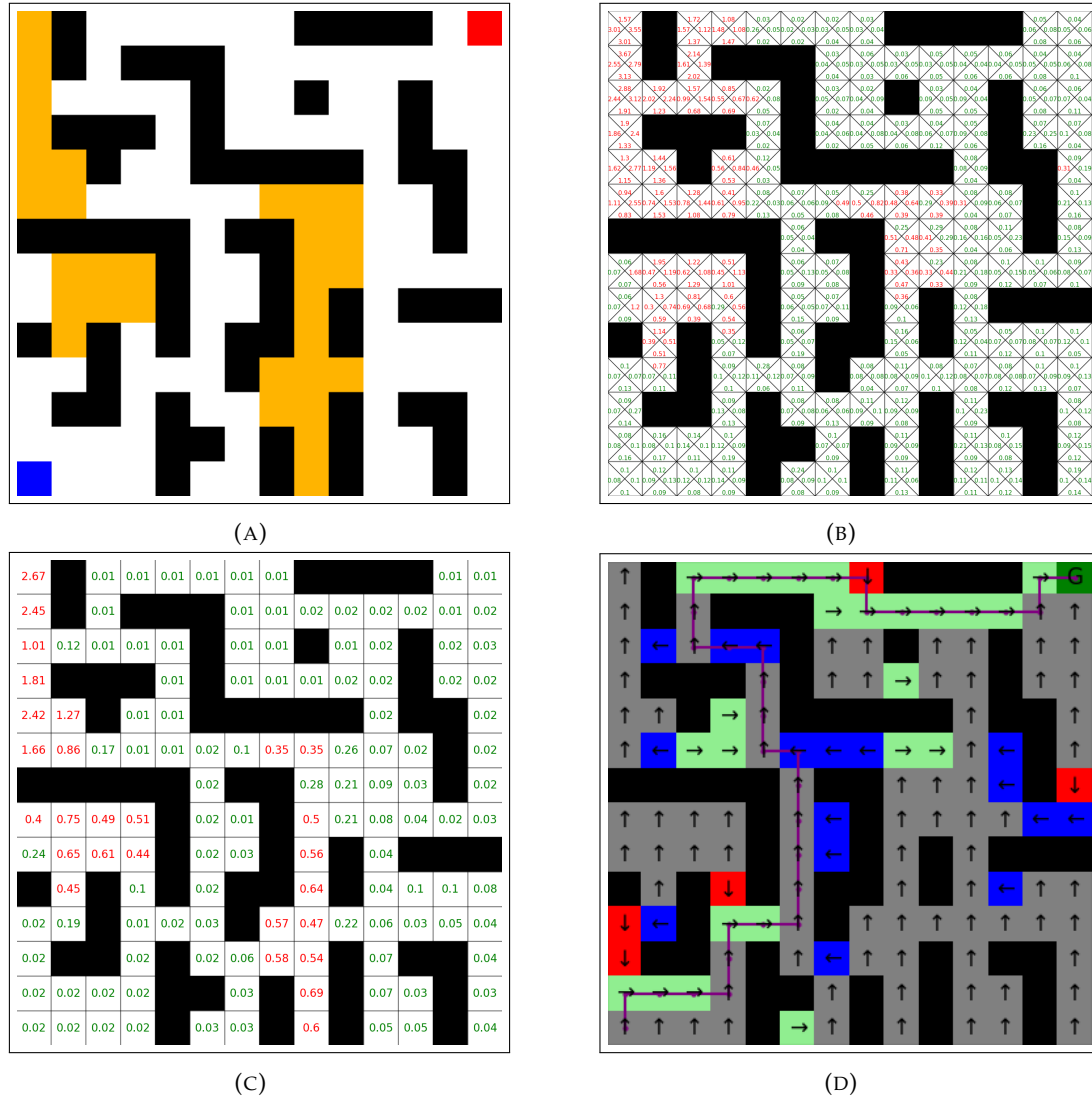


FIGURA 5.9: En (A) mapa del entorno con las casillas no demostradas en naranja. En (B) las incertidumbres dadas por el *ensemble* asociadas a cada acción y en (C) a cada estado. En (D) la política de un agente entrenado usando este *ensemble*, que evita correctamente los estados no demostrados.

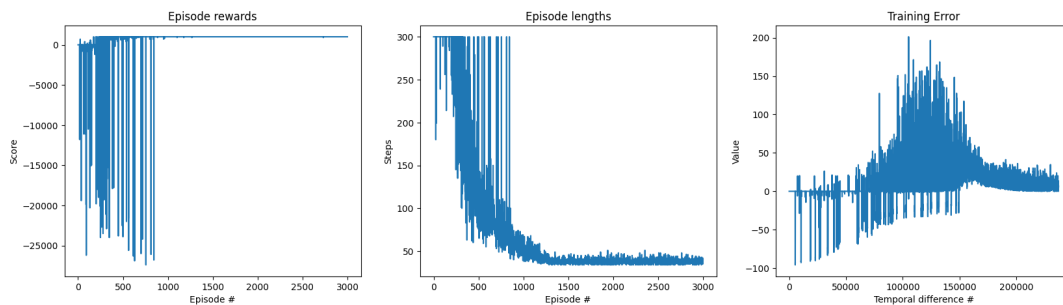


FIGURA 5.10: Estadísticas del agente entrenado (recompensa acumulada por episodio, pasos para completar el episodio y error de entrenamiento)

6 Conclusiones y trabajo futuro

El trabajo ha logrado avanzar en la investigación sobre aprendizaje por refuerzo *offline*, centrándose en la gestión de la incertidumbre para mejorar la robustez de modelos que permitan obtener una política segura a partir de un dataset recopilado de forma *offline*. Se han alcanzado en la práctica los objetivos propuestos teóricamente al comienzo del proyecto, incluyendo la generación de experiencias, la construcción de modelos con *ensembles* de redes neuronales, la medición de la incertidumbre de los estados y la integración de esta en la función de refuerzo.

Los resultados obtenidos de la experimentación en el dominio utilizado respaldan de forma concluyente la efectividad de la estrategia propuesta. Los agentes entrenados han demostrado ser capaces de evitar salirse de la distribución de datos del dataset *offline*, lo que les permite mitigar e incluso evitar el problema del *distributional drift*, que podría suponer que llegasen a situaciones potencialmente peligrosas al visitar estados que no habían sido demostrados previamente.

A pesar de los logros obtenidos, existen áreas de mejora y posibles direcciones para investigaciones futuras:

- **Exploración de técnicas avanzadas de incertidumbre:** La investigación y aplicación de técnicas más avanzadas para medir la incertidumbre, como la estimación de intervalos de confianza, podrían proporcionar *insights* adicionales y perfeccionar la capacidad del modelo para gestionar la incertidumbre.
- **Adaptación a otros dominios:** La extensión de la estrategia propuesta a dominios más complejos y realistas permitiría evaluar su generalización y aplicabilidad en escenarios del mundo real, ampliando así su campo de aplicación.
- **Adaptabilidad a entornos dinámicos:** Investigar métodos que permitan una adaptación más efectiva a entornos dinámicos y cambiantes, donde la distribución de datos puede variar con el tiempo.
- **Optimización de hiperparámetros:** Realizar un análisis más profundo de la configuración de hiperparámetros para optimizar el rendimiento del modelo en diferentes configuraciones de entorno.
- **Transferencia de conocimiento:** Explorar técnicas de transferencia de conocimiento para mejorar la eficiencia del aprendizaje por refuerzo *offline* mediante el aprovechamiento de los conocimientos ya obtenidos en este proyecto.
- **Escalabilidad:** Evaluar la escalabilidad del enfoque propuesto para manejar entornos más grandes y complejos, considerando la eficiencia computacional.

En resumen, el trabajo proporciona una base sólida para futuras investigaciones en el ámbito del aprendizaje por refuerzo *offline* y la gestión de la incertidumbre, e incluso podría servir como base para hacer pruebas en situaciones reales.

A Descripción del código

Todo el código y los modelos pueden ser descargados desde este repositorio de [GitHub](#) o bien desde [OneDrive](#).

Los programas principales replicar los experimentos de este TFG están incluidos en las carpetas `./src/` y `./src2/`, que contienen dos flujos de trabajo totalmente independientes.

Para instalar las librerías necesarias basta con instalar el contenido del *requirements.txt* con el comando `"pip install -r requirements.txt"` desde la terminal. Es recomendable crear un entorno virtual de 0 para realizar la instalación evitando problemas de incompatibilidad entre otras versiones de las librerías.

A continuación se describirá brevemente el contenido de cada una de las carpetas que componen el proyecto:

`./_discarded/`

Contiene pruebas fallidas en las que se intentaron usar librerías de RL previamente existentes, además de una primera versión del proyecto que estaba implementando en *TensorFlow*, antes de decidir migrar todo a *PyTorch*.

`./agentsClasses/`

Aquí están definidas las clases de los agentes DQN y DDQN (en *DQNClass.py*) y los Q-Learning (*QLearningClass.py*). También están las versiones de esas clases pero para interactuar con el entorno *offline* (*OfflineDQNClass.py* y *OfflineQLearningClass.py*).

`./data/csv/`

Contiene los dataset de las transiciones (generados con el primer entorno en *./src/1_CSVGeneration.ipynb*) de cada uno de los dos mapas que existen, antes (*history5x5.csv* y *history14x14.csv*) y después (*new5x5.csv* y *new14x14.csv*) de eliminar aquellas transiciones que se repitan más de un número X de veces.

`./data/EnvNNModels/`

Con los dataset de los *csv* se generan 2 modelos, un modelo de la dinámica del entorno y otro de la función de recompensa, para cada uno de los 2 mapas. En total son 4 redes *PyTorch* que se guardan en esta carpeta.

./data/json/

Aquí se guardan en formato *json* la información de los agentes *QLearning*, *DQN* y *DDQN* de los 2 mapas, entrenados con los programas *./src/4_1_QLearning.ipynb*, *./src/4_2_QLearningV2.ipynb*, *./src/5_1_DQNTrain.ipynb* y *./src/5_2_DQNTrainV2.ipynb*

Se guardan los siguientes datos de cada agente: ID, Episodios de entrenamiento, Máx. número de pasos por episodio, Algoritmo usado (*QL*, *DQN* o *DDQN*), Tamaño del *grid* (5x5 o 14x14), Coordenadas de la casilla inicial, Política entrenada, Tabla de valores de cada estado

./data/OfflineEnsembles/

Carpetas que contienen los *ensembles* de modelos para la predicción de la incertidumbre en los 2 mapas. Las carpetas *csv* dentro de las carpetas que contienen los modelos, guardan ficheros en formato *csv* con la información de la evolución de la pérdida de validación y de entrenamiento durante la fase de entrenamiento de los modelos.

La carpeta *5x5_5_model* se corresponde con el *ensemble* del apartado 5.2.1, *5x5_5_modelv2* con el del 5.2.2, *5_model* con el del 5.2.3 y *5_modelv2* con el del 5.2.4.

./data/txt/

Ficheros *txt* para debuguear más fácilmente los resultados de las predicciones de los *ensembles online*.

./envs/

Estos códigos contienen el entorno original SimpleGrid (*EnvCSV.py*), el que usa redes neuronales para modelar la dinámica del entorno y la función de recompensa (*EnvNN.py*) y el que usa como modelo el *ensemble offline* de redes neuronales (*EnvOffline.py*).

En el fichero *GridMaps.py* es donde están definidos como arrays de arrays de caracteres los mapas de los *grids*. Se pueden añadir o quitar fácilmente casillas no demostradas (aquellas que contienen un carácter 'x') para realizar experimentos, cambiar la configuración de los muros o la posición de las casillas de *Inicio* o *Meta*. También existe la posibilidad de crear otros nuevos mapas con distintas dimensiones.

Por último, hay un archivo que permite contiene las funciones que permiten instanciar los entornos usando los mapas (*CreateEnv.py*) y otro que contiene diferentes las diferentes arquitecturas de redes neuronales usadas a lo largo del proyecto (*NeuralNetworks.py*).

./img/

Contiene imágenes resultantes de algunos de los experimentos (gráficas de entrenamientos de agentes, modelos de entorno, políticas de comportamiento de agentes...)

./src/

Esta carpeta contiene los *notebooks* con los experimentos correspondientes con el primer enfoque del proyecto. Están ordenados alfabéticamente siguiendo el mismo orden secuencial con el que fueron creados y que debería seguir el proceso en caso de querer replicar los experimentos. A continuación se explicará el contenido de cada *notebook*:

1. `1_CSVGeneration.ipynb`: En este archivo, haciendo uso del entorno de `./envs/EnvCSV.py` se generan los dataset en formato *csv* con los que se entrenarán las redes neuronales para modelar la dinámica y la función de recompensa del entorno.
2. `2_NNModelsTrain.ipynb`: Aquí se entrenan las redes neuronales haciendo uso de los *csv* creados por el anterior *notebook*, para poder crear el entorno de `./envs/EnvNN.py`
3. `3_NNRun.ipynb`: Aquí se puede comprobar que el entorno implementado con las redes neuronales funciona correctamente.
4. `4_1_QLearning.ipynb` y `4_2_QLearningV2.ipynb`: Programas para entrenar agentes de *QLearning* que sean capaces de resolver el problema de ir desde el Inicio a la Meta usando el entorno `./envs/EnvNN.py`, de forma que se obtengan trayectorias que puedan usarse como "demostradores". La estructura del `4_2` es algo distinta a la de `4_1`, para permitir entrenar muchos más agentes de cada vez y con distintas características de número de episodios y máximo número de pasos por episodio. Los resultados se guardan en los ficheros *json* de `./data/json/`
5. `5_1_DQNTrain.ipynb` y `5_2_DQNTrainV2.ipynb`: Lo mismo que los *notebooks* del punto anterior, pero para agentes *DQN* y *DDQN*.
6. `6_JsonManagement.ipynb`: Este programa permite visualizar y modificar la información de los *json* creados en los *notebooks* previos.
7. `7_OfflinesFamiliesModels.ipynb`: En este *notebook* me di cuenta de que el enfoque que estaba usando no me iba a dar resultados congruentes, por lo que decidí abandonarlo y pasar al enfoque de `./src2/`. Como en cada mapa existe una serie de casillas de inicio definidas, entre las que se escoge una de ellas de forma aleatoria cada vez que se instancia el entorno, la idea era juntar los demostradores en grupos según su casilla de inicio. Así, con las trayectorias de cada grupo se generaría un modelo, y al juntar todos se podría obtener un modelo global que permitiese a otro agente resolver de cero el problema de llegar del Inicio a la Meta.

./src2/

Esta carpeta se corresponde con el enfoque final del proyecto y, como la anterior, sus *notebooks* también están ordenados secuencialmente. A continuación, una breve explicación de cada uno:

1. `1_OfflineEnsemblesGenerator.ipynb`: En este *notebook*, a partir de los mapas definidos en `./envs/GridMaps` como arrays de arrays de caracteres, se generan todas las transiciones posibles. La clave es que podemos elegir casillas, en las

que pondremos el caracter 'x', para que sean casillas no demostradas. Con esas transiciones, el dataset "D", se crean los modelos *offline*, un *ensemble* de modelos (inicializados con distinto *seed*, i.e. distinto vector de parámetros o pesos iniciales). Haciendo uso de estos *ensembles* en *EnvOffline.py* obtenemos un entorno en el que seremos capaces de discernir la incertidumbre entre los distintos estados, de manera que, aquellos que no habían sido incluidos en el dataset (los no demostrados 'x') tengan una incertidumbre mucho mayor que los que si. Así, penalizandolos cuando lleguen a estos estados, podremos entrenar agentes que consigan resolver el entorno mediante rutas que eviten pasar por ninguno de los estados no demostrados.

2. *2_Experiments.ipynb*: Aquí se realizan una serie de pruebas descritas en la memoria para probar a aplicar distintas técnicas (basadas en el *bagging*) sobre el dataset generado anteriormente, y ver como afectaría a la eficacia de los *ensembles* generados con cada una.
3. *3_Visualization.ipynb*: Con este *notebook* se pueden ver las gráficas de las pérdidas, tanto de validación como de entrenamiento, de los modelos que componen los *ensembles* durante su fase de entrenamiento. También se pueden ver los resultados de incertidumbre que nos dan los *ensembles* para cada casilla (y, si se quiere, para cada acción en cada una de ellas por separado) del *grid* gráficamente.
4. *4_OfflineQLearning.ipynb* y *5_OfflineDQNTrain.ipynb*: Haciendo uso de los *ensembles* para modelar el entorno desde *EnvOffline.py*, podemos conseguir que agentes resuelvan el entorno evitando las casillas no demostradas, tanto con *QLearning* como con *DQN* o *DDQN*, para los dos mapas.

./utilities/

Programas que contienen funciones auxiliares para el resto:

1. *jsonRW.py*: Contiene funciones que permiten acceder y visualizar, así como editar la información contenidas en los archivos *json* de *./data/json/* desde el programa *./src/6_JsonManagement.ipynb*
2. *plots.py*: funciones para plotear las políticas y trayectorias de los agentes entrenados en los programas *./src/4_1_QLearning.ipynb*, *./src/4_2_QLearningV2.ipynb*, *./src/5_1_DQNTrain.ipynb* y *./src/5_2_DQNTrainV2.ipynb*.
3. *transitionDatasetGeneration.py*: A pesar de su nombre, además de contener funciones para generar los dataset para crear los modelos del entorno *offline*, también incluyen todo tipo de funciones auxiliares usadas en todos los programas de *./src2/*, como pueden ser la clase para el entrenamiento de los *ensembles* de modelos o las funciones que permiten visualizar el mapa completo con la incertidumbre de cada estado.

Estructura de carpetas

```
./
|-- _discarded/
|-- agentsClasses/
|   |-- DQNClass.py
|   |-- QLearningClass.py
|   |-- OfflineDQNClass.py
|   |-- OfflineQLearningClass.py
|-- data/
|   |-- csv/
|   |-- EnvNNModels/
|   |-- json/
|   |-- OfflineEnsembles/
|   |-- txt/
|-- envs/
|   |-- EnvCSV.py
|   |-- EnvNN.py
|   |-- EnvOffline.py
|   |-- GridMaps.py
|   |-- CreateEnvs.py
|   |-- NeuralNetworks.py
|-- img/
|-- src/
|   |-- 1_CSVGeneration.ipynb
|   |-- 2_NNModelsTrain.ipynb
|   |-- 3_NNRun.ipynb
|   |-- 4_1_QLearning.ipynb
|   |-- 4_2_QLearningV2.ipynb
|   |-- 5_1_DQNTrain.ipynb
|   |-- 5_2_DQNTrainV2.ipynb
|   |-- 6_JsonManagement.ipynb
|   |-- 7_OfflinesFamiliesModels.ipynb
|-- src2/
|   |-- 1_OfflineEnsemblesGenerator.ipynb
|   |-- 2_Experiments.ipynb
|   |-- 3_Visualization.ipynb
|   |-- 4_OfflineQLearning.ipynb
|   |-- 5_OfflineDQNTrain.ipynb
|-- utilities/
|   |-- jsonRW.py
|   |-- plots.py
|   |-- transitionDatasetGeneration.py
|-- README.md
|-- requeriments.txt
```


B Resultados de los agentes demostradores

En este apéndice se encuentran los resultados obtenidos durante el primer enfoque del proyecto, en el que se pretendía generar el dataset *offline* haciendo uso de agentes de tipo *QLearning*, *DQN* y *DDQN* a modo de demostradores. Estos agentes se construyen siguiendo una filosofía de métodos basados en el modelo, por lo que primero se entrenan 2 redes neuronales con ese fin, una para simular la dinámica del entorno y otra la función de recompensa. Haciendo uso de esas redes, se entrena a los agentes para lograr llegar de la casilla de *Inicio* a la de *Meta*, generando trayectorias o demostraciones cuyas transiciones se usan para construir el dataset *offline*.

B.1. Creación de los modelos para los demostradores

Para los experimentos realizados durante este trabajo se ha decidido emplear 2 configuraciones diferentes de entornos, que ya se presentaron gráficamente en el Capítulo 3 (Figuras 3.1 y 3.2). La primera de ellas tiene 5 filas y 5 columnas y la segunda tiene un tamaño mayor, con 14 filas y 14 columnas. En ambas se puede ver que la *Meta*, representada por las celdas de color rojo, se encuentra en la posición superior derecha, mientras que la casilla de *Inicio*, de color verde, se randomiza cada vez que se instancia un entorno entre una serie de estados prefijados posibles.

Con el objetivo de conseguir crear un dataset *offline*, se pretende construir agentes demostradores que basen su comportamiento en modelos. Por ello, se van a entrenar para cada uno de los mapas 2 redes neuronales con *PyTorch* que serán los modelos para estos agentes demostradores. Una de las redes debe simular la dinámica del entorno, por lo que recibirá como entrada un tensor con 3 elementos con la información del estado s_t y la acción a_t , con la forma $[y_t, x_t, a_t]$, y a su salida devolverá el tensor $[y_{t+1}, x_{t+1}]$, con la información del estado siguiente s_{t+1} . La otra red sustituirá a la función de recompensa, para lo que tomará como entrada el mismo tensor que la primera, pero en su salida será un tensor con un solo elemento $[r_{t+1}]$.

Debido a la diferencia de tamaño y, por tanto, de complejidad de los 2 mapas diseñados, la arquitectura de las redes no puede ser la misma, pero si que todas las capas usan activación ReLU. Las arquitecturas empleadas son las siguientes (entre paréntesis el número de neuronas en esa capa):

- 5x5:

- Dinámica del entorno: $Input(3) \rightarrow HiddenLayer(64) \rightarrow Output(2)$
- Recompensa: $Input(3) \rightarrow HiddenLayer(64) \rightarrow Output(1)$

■ **14x14:**

- Dinámica: $Input(3) \rightarrow HiddenLayer(256) \rightarrow HiddenLayer(128) \rightarrow Output(2)$
- Recompensa: $Input(3) \rightarrow HiddenLayer(64) \rightarrow HiddenLayer(64) \rightarrow Output(1)$

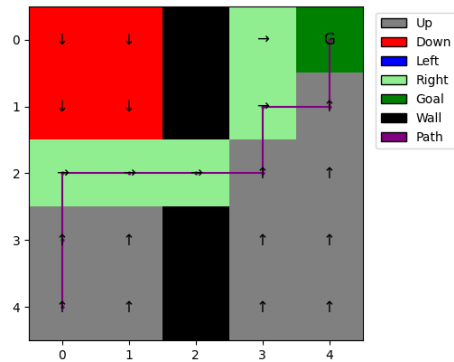
Para poder entrenar estas redes se necesita crear un dataset por mapa, para lo que se simulan 100 episodios en cada uno. Para ello se hace uso de agentes en el entorno *OpenAI Gym* implementado en 3.3, que samplean aleatoriamente alguna de las acciones $a \in \mathcal{A}$ para posteriormente pasarla como parámetro a la función *step* del entorno. A cada paso que da el agente, se almacena la información de las tuplas de las transiciones generadas, guardándolas en su respectivo archivo en formato *csv* para poder usarlas para entrenar a los modelos que usarán después los demostradores.

Siguiendo este método, se logra crear redes que tienen una pérdida totalmente despreciable, aunque sus predicciones son valores reales y tanto los estados como la recompensa del entorno deben ser valores enteros, pero basta con hacer un sencillo redondeo.

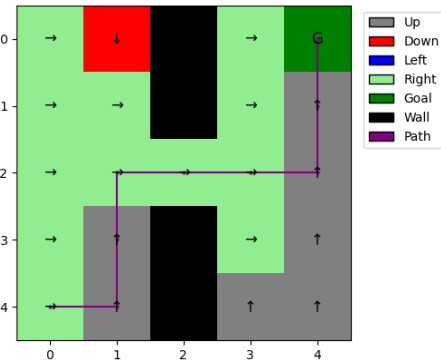
B.2. Trayectorias de agentes demostradores

En esta sección, se muestran algunos de los agentes *QLearning*, *DQN* y *DDQN* entrenados durante el proyecto con el fin de ser empleados como demostradores que permitiesen la creación del dataset *offline* con sus experiencias, antes de decidir crearlo de manera sintética con el Algoritmo 2. Esta información se encuentra almacenada en archivos en formato *json* y puede ser recuperada para su visualización, como se explica en el Apéndice A.

Agent 102 - Q-Learning Start pos: [4 0] Episodes: 5000 Max steps: 0

(A) Agente *QLearning*

Agent 2 - DQN Start pos: [4 0] Episodes: 500 Max steps: 25

(B) Agente *DQN*

Agent 5 - DDQN Start pos: [4 0] Episodes: 500 Max steps: 25

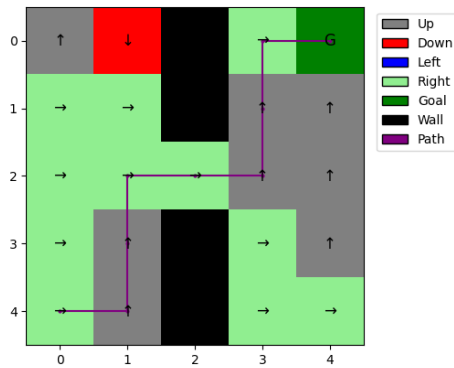
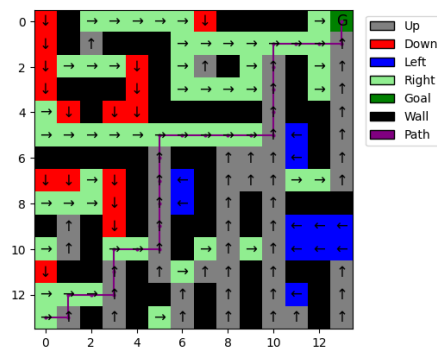
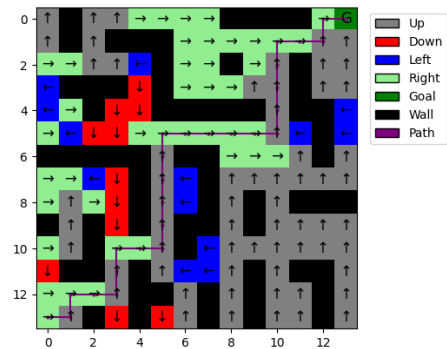
(C) Agente *DDQN*

FIGURA B.1: Ejemplos de demostradores en el entorno de 5x5

Agent 2 - Q-Learning Start pos: [13 0] Episodes: 15000 Max steps: 0

(A) Agente *QLearning*

Agent 31 - DQN Start pos: [13 0] Episodes: 3000 Max steps: 150

(B) Agente *DQN*

Agent 13 - DDQN Start pos: [13 0] Episodes: 3000 Max steps: 150

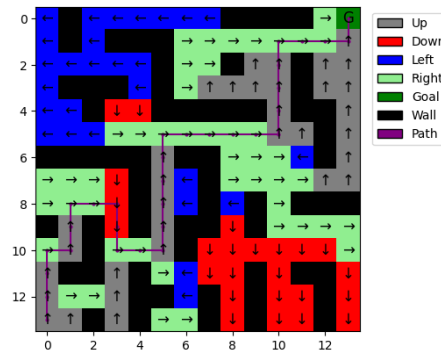
(C) Agente *DDQN*

FIGURA B.2: Ejemplos de demostradores en el entorno de 14x14

Bibliografía

- [1] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 1st edition. MIT Press Cambridge, MA, USA, 1998.
- [2] D. Silver, T. Hubert, J. Schrittwieser et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, 2017. arXiv: [1712.01815 \[cs.AI\]](#).
- [3] I. B. Oriol Vinyals y D. S. et al., «Grandmaster level in starcraft II using multi-agent reinforcement learning,» *Nature*, vol. 366, págs. 350-354, 2019. dirección: <https://www.nature.com/articles/s41586-019-1724-z>.
- [4] S. Levine, A. Kumar, G. Tucker y J. Fu, *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*, 2020. arXiv: [2005.01643 \[cs.LG\]](#).
- [5] P. S. Thomas, «Safe reinforcement learning,» Tesis doct., University of Massachusetts Amherst, 2014.
- [6] P. S. Thomas, B. C. da Silva, A. G. Barto, S. Giguere, Y. Brun y E. Brunskill, «**Preventing Undesirable Behavior of Intelligent Machines**,» *Science*, vol. 366, n.º 6468, págs. 999-1004, 22, DOI: [10.1126/science.aag3311](#), ISSN: 0036-8075. DOI: [10.1126/science.aag3311](#).
- [7] T. G. Sascha Lange y M. A. Riedmiller, *Reinforcement Learning. Adaptation, Learning, and Optimization*, vol 12. Springer, 2012, cap. Batch reinforcement learning.
- [8] T. Yu, G. Thomas, L. Yu et al., *MOPO: Model-based Offline Policy Optimization*, 2020. arXiv: [2005.13239 \[cs.LG\]](#).
- [9] R. Kidambi, A. Rajeswaran, P. Netrapalli y T. Joachims, *MOREL : Model-Based Offline Reinforcement Learning*, 2021. arXiv: [2005.05951 \[cs.LG\]](#).
- [10] J. L. Pérez, J. Corrochano, J. García et al., «Discrete Uncertainty Quantification For Offline Reinforcement Learning,» *Journal of Artificial Intelligence and Soft Computing Research*, vol. 13, n.º 4, págs. 273-287, 2023.
- [11] S. Bhatt. «Reinforcement Learning 101.» (2018), dirección: <https://towardsdatascience.com/reinforcement-learning-101-e24b50e1d292>.
- [12] R. Bellman, «Dynamic programming and stochastic control processes,» *Information and Control*, vol. 1, n.º 3, págs. 228-239, 1958, ISSN: 0019-9958. DOI: [https://doi.org/10.1016/S0019-9958\(58\)80003-0](https://doi.org/10.1016/S0019-9958(58)80003-0). dirección: <https://www.sciencedirect.com/science/article/pii/S0019995858800030>.
- [13] M. G. P. Poupart e Y. Engel. «Slide Presentation: RL Tutorial.» (2011), dirección: <https://slideplayer.com/slide/8275909/>.
- [14] D. P. Bertsekas y J. N. Tsitsiklis, *Neuro-dynamic programming*. (Optimization and neural computation series). Athena Scientific, 1996, vol. 3, págs. I-XIII, 1-491, ISBN: 1886529108.
- [15] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

- [16] R. Sutton y M. Kaufmann, «Planning by incremental dynamic programming,» en *Proceedings of the Eighth International Workshop on Machine Learning*, p. 353–357, 1991. dirección: <http://incompleteideas.net/papers/sutton-91a.pdf>.
- [17] M. P. Deisenroth y C. E. Rasmussen, «PILCO: A Model-Based and Data-Efficient Approach to Policy Search,» en *Proceedings of the 28th International Conference on Machine Learning*, ép. ICML'11, Madison, WI, USA: Omnipress, 2011, 465–472, ISBN: 9781450306195.
- [18] G. A. Rummery y M. Niranjan, «On-Line Q-Learning Using Connectionist Systems,» Cambridge University Engineering Department, Cambridge, England, inf. téc. TR 166, 1994.
- [19] C. J. C. H. Watkins, «Learning from Delayed Rewards,» Tesis doct., King's College, Oxford, 1989.
- [20] V. Mnih, K. Kavukcuoglu, D. Silver et al., «Human-level control through deep reinforcement learning,» *Nature*, vol. 518, n.º 7540, págs. 529-533, 2015. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236).
- [21] T. P. Lillicrap, J. J. Hunt, A. Pritzel et al., *Continuous control with deep reinforcement learning*, 2019. arXiv: [1509.02971](https://arxiv.org/abs/1509.02971) [cs.LG].
- [22] H. He, *A Survey on Offline Model-Based Reinforcement Learning*, 2023. arXiv: [2305.03360](https://arxiv.org/abs/2305.03360) [cs.LG].
- [23] S. Levine. «Supervised Learning of Behaviors.» (2018), dirección: <http://rail.eecs.berkeley.edu/deeprlcourse/static/slides/lec-2.pdf>.
- [24] G. Brockman, V. Cheung, L. Pettersson et al., *OpenAI Gym*, 2016. arXiv: [1606.01540](https://arxiv.org/abs/1606.01540) [cs.LG].
- [25] D. Schwab. «Deep RL and Controls OpenAI Gym Recitation.» (2017), dirección: https://katefvision.github.io/10703_openai_gym_recitation.pdf.
- [26] M. Towers, J. K. Terry, A. Kwiatkowski et al., *Gymnasium by The Farama Foundation*, <https://gymnasium.farama.org/>, 2022.
- [27] L. D'Amato, *Simple Gridworld Environment for OpenAI Gym*, <https://github.com/damat-le/gym-simplegrid>, 2022.
- [28] A. Paszke y M. Towers, *Reinforcement Learning (DQN) Tutorial*, https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html, 2017.
- [29] U. Singh, *Deep Q-Network with Pytorch*, <https://unnatsingh.medium.com/deep-q-network-with-pytorch-d1ca6f40bfda>, 2019.
- [30] A. Paszke, S. Gross, F. Massa et al., «PyTorch: An Imperative Style, High-Performance Deep Learning Library,» en *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, págs. 8024-8035. dirección: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [31] L. Breiman, «Bagging Predictors,» *Machine Learning*, vol. 24, n.º 2, págs. 123-140, 1996.